

Estudo Dirigido de Processamento Digital de Sinais – Parte 1

Sinais Discretos, Amostragem, Sistemas LTI e Convolução

Alunas: Aicha Nacambo e Katia Emmanuelle Gomes da Silva

IFPB Campus Campina Grande · Prof. Moacy Pereira da Silva · 2026.2

1. Introdução

Este relatório apresenta a Parte 1 do Estudo Dirigido de Processamento Digital de Sinais (PDS), do IFPB Campus Campina Grande, período 2026.2, sob orientação do professor Moacy Pereira da Silva.

O objetivo da Parte 1 é construir o caminho que leva de um fenômeno físico até um sinal processado por um sistema digital:

$$\text{Fenômeno físico} \rightarrow x(t) \rightarrow \text{Amostragem} \rightarrow x[n] \rightarrow \text{Sistema LTI} \rightarrow y[n]$$

O relatório está organizado na ordem do roteiro:

- **Tópicos 1 a 5 (Seções 2 a 6):** sinais contínuos e discretos, amostragem, quantização, sequências fundamentais e operações com sinais.
- **Tópicos 6 a 10 (Seções 7 a 11):** energia e potência, sistemas discretos, sistemas LTI, convolução discreta e convolução como filtragem.
- **Mini Projeto Integrador (Seção 12):** simulação de uma cadeia completa de aquisição e processamento da vibração de um motor, que aplica os conceitos dos dez tópicos.
- **Questões de Síntese, Conclusões e Referências (Seções 13 a 15).**
- **README e códigos (Seção 16 e Apêndice A).**

Cada tópico segue a estrutura pedida no roteiro: resumo conceitual, formulação matemática, exemplo, simulação, resultados e discussão. Todas as simulações foram feitas em Python, com NumPy e Matplotlib, e podem ser executadas no Google Colab. Os códigos usam semente fixa no gerador de números aleatórios, de modo que os gráficos e os valores numéricos deste relatório podem ser reproduzidos.

2. Tópico 1 – Sinais Contínuos e Discretos

2.1 Resumo conceitual

Um sinal é uma função que carrega informação sobre o comportamento de uma grandeza. Na engenharia, essa grandeza quase sempre vem de um fenômeno físico (temperatura, pressão, som, tensão elétrica) e a variável independente mais comum é o tempo.

A diferença entre sinal contínuo e sinal discreto está na variável independente, e não na amplitude:

- **Sinal contínuo, $x(t)$:** existe para todo instante t de um intervalo. Entre dois instantes quaisquer sempre há infinitos outros valores definidos.
- **Sinal discreto, $x[n]$:** existe apenas para valores inteiros de n . Entre $n = 3$ e $n = 4$ o sinal não vale zero: ele simplesmente não está definido.

Exemplos reais de sinais contínuos:

- a tensão elétrica na saída de um microfone analógico;
- a temperatura de um ambiente, que varia sem interrupção ao longo do dia;
- a pressão dentro de uma tubulação industrial.

Exemplos reais de sinais discretos:

- as leituras horárias de temperatura registradas por uma estação meteorológica;
- as amostras de um arquivo de áudio digital, gravadas 44.100 vezes por segundo;
- o consumo mensal de energia de uma residência, um valor por mês.

2.2 Formulação matemática

Representar um fenômeno matematicamente significa escolher uma função cujos parâmetros tenham significado físico. Um modelo muito usado é a senoide:

$$x(t) = A \cos(2\pi f_0 t + \phi), \quad t \in \mathbb{R}$$

Nesse modelo, A é a amplitude (o maior afastamento em relação à média), f_0 é a frequência em hertz (quantas oscilações por segundo) e ϕ é a fase em radianos (o deslocamento da curva no tempo).

O sinal discreto correspondente é uma sequência de números indexada por um inteiro:

$$x[n], \quad n \in \mathbb{Z}$$

Quando a sequência vem da observação de um sinal contínuo a cada T_s segundos, vale $x[n] = x(nT_s)$. Essa relação é o assunto do Tópico 2.

2.3 Exemplos

Foram usados dois exemplos, cada um na forma contínua e na forma discreta.

Exemplo	Forma contínua	Forma discreta
Cosseno de 2 Hz	$x(t) = \cos(2\pi \cdot 2 \cdot t)$, com t de 0 a 1 s	$x[n] = \cos(2\pi \cdot 2 \cdot n \cdot T_s)$, com $T_s = 0,05$ s e n de 0 a 20
Temperatura de um ambiente	$T(t) = 26 + 5 \cos(2\pi(t - 15)/24)$, com t em horas	$T[n]$, uma leitura por hora, n de 0 a 23

No modelo de temperatura, 26 °C é a média do dia, 5 °C é a amplitude da variação, 24 h é o período e o deslocamento de 15 h coloca o máximo às 15 h.

2.4 Simulação

Um computador não armazena um sinal contínuo, pois isso exigiria infinitos valores. O sinal contínuo foi simulado com uma grade muito fina de 5.000 pontos, desenhada com linha. O sinal discreto foi desenhado com hastes (stem), uma por amostra.

Código completo (arquivo `simulacoes/sinais/topico1_sinais.py`):

```
"""Tópico 1 - Sinais contínuos e discretos."""
import os
import numpy as np
import matplotlib.pyplot as plt

PASTA = "figuras"
os.makedirs(PASTA, exist_ok=True)
plt.rcParams.update({"font.size": 10, "axes.grid": True, "grid.alpha": 0.3})

# Exemplo 1: cosseno de 2 Hz. O "contínuo" é simulado com uma grade muito fina.
f0 = 2.0
t = np.linspace(0, 1, 5000)
x_t = np.cos(2 * np.pi * f0 * t)

# O mesmo cosseno observado a cada Ts = 1/20 s
fs = 20.0
Ts = 1 / fs
n = np.arange(0, 21)
x_n = np.cos(2 * np.pi * f0 * n * Ts)

# Exemplo 2: temperatura de um ambiente ao longo de um dia (máximo às 15 h)
h = np.linspace(0, 24, 5000)
temp_t = 26 + 5 * np.cos(2 * np.pi * (h - 15) / 24)

# Leitura de um sensor a cada hora
k = np.arange(0, 24)
temp_n = 26 + 5 * np.cos(2 * np.pi * (k - 15) / 24)

fig, ax = plt.subplots(2, 2, figsize=(11, 6.5))

ax[0, 0].plot(t, x_t, color="tab:blue")
ax[0, 0].set(title="Sinal contínuo:  $x(t) = \cos(2\pi \cdot 2 \cdot t)$ ", xlabel="Tempo t (s)", ylabel="Amplitude")

ax[0, 1].stem(n, x_n, linefmt="tab:red", markerfmt="o", basefmt=" ")
ax[0, 1].set(title="Sinal discreto:  $x[n] = \cos(2\pi \cdot 2 \cdot n \cdot T_s)$ ,  $T_s = 0,05$  s",
             xlabel="Índice n (amostras)", ylabel="Amplitude", xticks=np.arange(0, 21, 2))

ax[1, 0].plot(h, temp_t, color="tab:blue")
ax[1, 0].set(title="Temperatura contínua T(t)", xlabel="Tempo t (h)", ylabel="Temperatura (°C)")

ax[1, 1].stem(k, temp_n, linefmt="tab:red", markerfmt="o", basefmt=" ")
ax[1, 1].set(title="Temperatura discreta T[n], uma leitura por hora",
             xlabel="Índice n (horas)", ylabel="Temperatura (°C)", ylim=(20, 32))

fig.tight_layout()
fig.savefig(os.path.join(PASTA, "t1_continuo_discreto.png"), dpi=150)
plt.show()

print("Pontos usados para simular x(t):", t.size)
print("Amostras de x[n]:", n.size)
print("x[n] para n = 0..5:", np.round(x_n[:6], 4))
print("T[n] às 3 h, 9 h, 15 h e 21 h:", np.round(temp_n[[3, 9, 15, 21]], 2))
```

2.5 Resultados

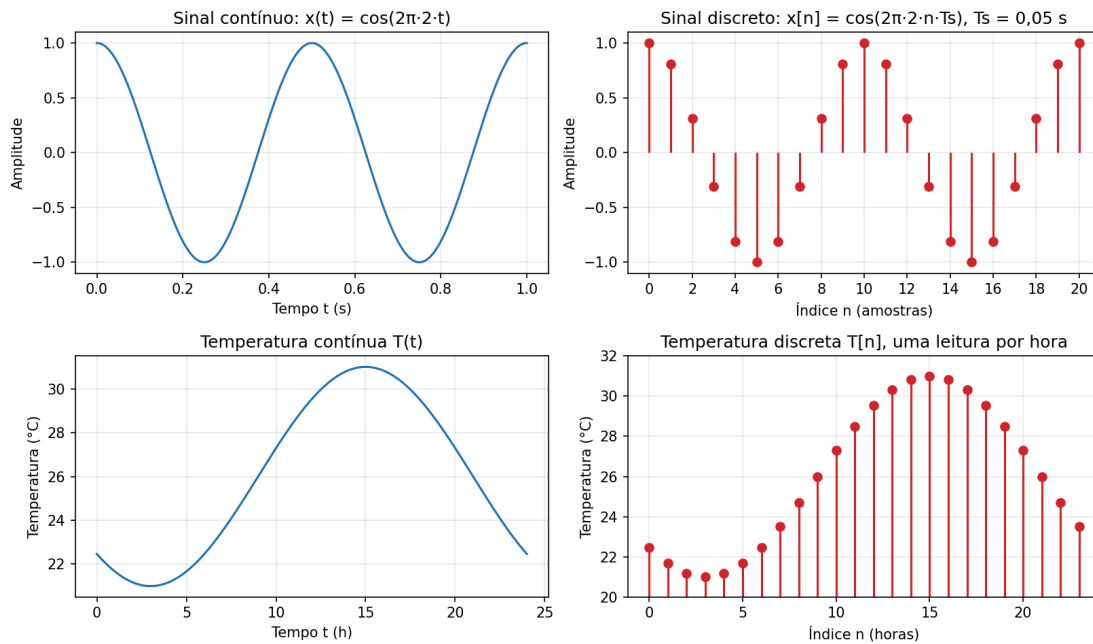


Figura 1. Sinais contínuos (à esquerda) e discretos (à direita) dos dois exemplos.

Valores numéricos obtidos:

Grandeza	Valor
Pontos usados para simular $x(t)$	5.000
Amostras de $x[n]$	21
$x[n]$ para $n = 0$ a 5	1,000; 0,809; 0,309; -0,309; -0,809; -1,000
$T[n]$ às 3 h, 9 h, 15 h e 21 h	21 °C; 26 °C; 31 °C; 26 °C

2.6 Discussão

Os gráficos da esquerda são curvas sem interrupção, enquanto os da direita são conjuntos de pontos isolados. Essa é a diferença central: $x(t)$ responde à pergunta “quanto vale o sinal em qualquer instante?”, e $x[n]$ responde apenas “quanto vale o sinal na amostra n ?”.

O eixo horizontal também muda de significado. No sinal contínuo ele está em segundos ou horas. No sinal discreto ele é apenas um índice inteiro, e o tempo físico só é recuperado quando se conhece T_s , pela relação $t = nT_s$. Na Figura 1, a amostra $n = 5$ do cosseno corresponde a $t = 5 \times 0,05 = 0,25$ s, exatamente o instante do primeiro mínimo de $x(t)$, e por isso $x[5] = -1$.

As amostras coincidem com a curva contínua nos instantes em que foram tomadas, pois $x[n] = x(nT_s)$. O cosseno de 2 Hz tem período de 0,5 s, que corresponde a $0,5/0,05 = 10$ amostras. Isso aparece no gráfico discreto: o padrão se repete a cada 10 hastes.

No exemplo da temperatura, as 24 leituras horárias acompanham bem a curva porque a temperatura varia devagar em relação ao intervalo de uma hora. O resultado era esperado: o mínimo de 21 °C ocorre às 3 h e o máximo de 31 °C às 15 h, como define o modelo.

Uma limitação da simulação é que o sinal “contínuo” do computador também é discreto, apenas com uma grade muito mais fina. Ele é uma aproximação visual suficiente para comparar as duas representações.

3. Tópico 2 – Amostragem

3.1 Resumo conceitual

Amostrar um sinal é medir o seu valor em instantes igualmente espaçados e guardar apenas essas medidas. Fisicamente, é o que faz um conversor analógico-digital: a cada T_s segundos ele “fotografa” a grandeza e registra um número. Tudo o que acontece entre duas fotografias é perdido.

O intervalo entre duas amostras é o período de amostragem T_s , em segundos. O número de amostras tomadas por segundo é a frequência de amostragem f_s , em hertz. As duas grandezas são inversas: quanto menor o intervalo entre as amostras, mais amostras cabem em um segundo.

3.2 Formulação matemática

A sequência discreta é obtida avaliando o sinal contínuo nos instantes de amostragem:

$$x[n] = x(nT_s), \quad t_n = nT_s, \quad f_s = \frac{1}{T_s}$$

Para uma senoide de frequência f_0 , a amostragem resulta em:

$$x[n] = \cos(2\pi f_0 n T_s) = \cos\left(2\pi \frac{f_0}{f_s} n\right) = \cos(\omega_0 n), \quad \omega_0 = 2\pi \frac{f_0}{f_s}$$

A sequência depende apenas da razão f_0/f_s . O inverso dessa razão, f_s/f_0 , é o número de amostras tomadas em cada período da senoide.

3.3 Exemplo

Foi usada a senoide $x(t) = \cos(2\pi \cdot 5 \cdot t)$, com $f_0 = 5$ Hz e período de 0,2 s, observada durante 0,8 s (quatro períodos). Ela foi amostrada com três frequências: 100 Hz, 25 Hz e 8 Hz.

3.4 Simulação

Código completo (arquivo `simulacoes/amostragem/topico2_amostragem.py`):

```
"""Tópico 2 - Amostragem de uma senoide com três frequências de amostragem."""
import os
import numpy as np
import matplotlib.pyplot as plt

PASTA = "figuras"
os.makedirs(PASTA, exist_ok=True)
plt.rcParams.update({"font.size": 10, "axes.grid": True, "grid.alpha": 0.3})

# Senoide de 5 Hz observada durante quatro períodos
```

```

f0 = 5.0
duracao = 0.8
t = np.linspace(0, duracao, 4000)
x_t = np.cos(2 * np.pi * f0 * t)

frequencias = [100.0, 25.0, 8.0]
fig, ax = plt.subplots(3, 1, figsize=(10, 9), sharex=True)

for eixo, fs in zip(ax, frequencias):
    Ts = 1 / fs
    n = np.arange(0, int(round(duracao * fs)) + 1)
    t_n = n * Ts
    x_n = np.cos(2 * np.pi * f0 * t_n)

    eixo.plot(t, x_t, color="tab:blue", label="x(t) contínuo, f0 = 5 Hz")
    eixo.stem(t_n, x_n, linefmt="tab:red", markerfmt="o", basefmt=" ", label="amostras x[n]")

    # Com fs = 8 Hz as amostras também pertencem a um cosseno de 3 Hz
    if fs == 8.0:
        eixo.plot(t, np.cos(2 * np.pi * 3 * t), "--", color="gray", label="cosseno de 3 Hz")
        diferenca = np.max(np.abs(x_n - np.cos(2 * np.pi * 3 * t_n)))
        print("Diferença máxima entre as amostras e o cosseno de 3 Hz:", f"{diferenca:.2e}")

    eixo.set(title=f"fs = {fs:g} Hz, Ts = {Ts * 1000:g} ms, {fs / f0:g} amostras por período",
            ylabel="Amplitude")
    eixo.legend(loc="upper right", fontsize=8)
    print(f"fs = {fs:6.1f} Hz | Ts = {Ts * 1000:6.1f} ms | amostras = {n.size:3d} | "
          f"amostras por período = {fs / f0:.1f}")

ax[-1].set_xlabel("Tempo t (s)")
fig.tight_layout()
fig.savefig(os.path.join(PASTA, "t2_amostragem.png"), dpi=150)
plt.show()

```

3.5 Resultados

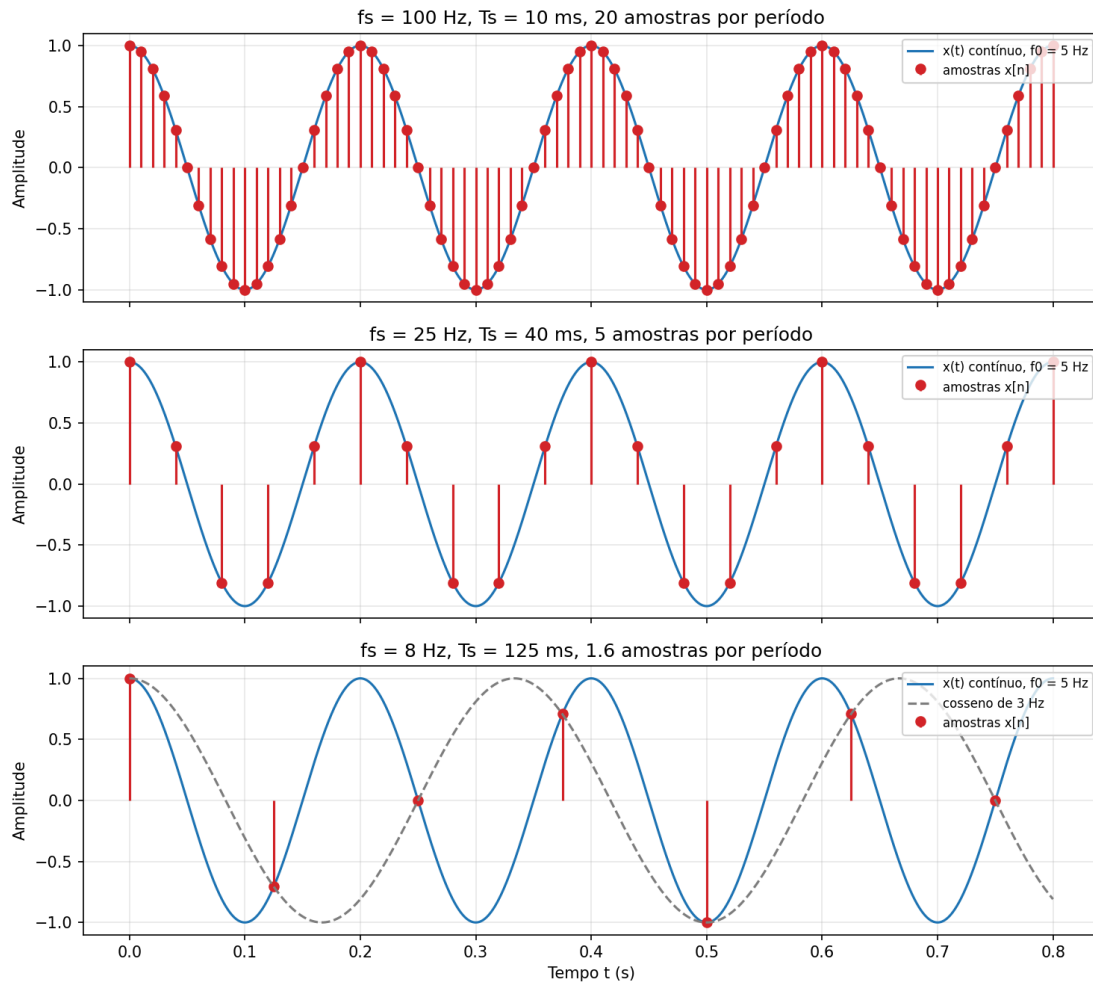


Figura 2. Senoide de 5 Hz (linha azul) e suas amostras (hastes vermelhas) para três frequências de amostragem.

f_s (Hz)	T_s (ms)	Amostras em 0,8 s	Amostras por período (f_s/f_0)
100	10	81	20
25	40	21	5
8	125	7	1,6

3.6 Discussão

Em todos os casos as hastes tocam a curva azul, pois cada amostra é o valor exato de $x(t)$ no instante nT_s . O que muda de um gráfico para o outro é a quantidade de informação que sobra entre as amostras. O único parâmetro alterado foi f_s .

Com $f_s = 100$ Hz há 20 amostras por período. Olhando apenas as hastes, sem a curva azul, a forma da senoide é reconhecida de imediato: amplitude, período e posição dos máximos estão claros.

Com $f_s = 25$ Hz há 5 amostras por período. A sequência ainda se repete a cada 5 hastes e o máximo aparece (amostras em $t = 0; 0,2; 0,4$ s), mas o mínimo de -1 nunca é amostrado. O menor valor registrado é $\cos(2\pi \cdot 2/5) = -0,809$. A representação é mais pobre, embora o ritmo da oscilação ainda esteja correto.

Com $f_s = 8$ Hz há apenas 1,6 amostra por período. As 7 amostras não acompanham mais a oscilação de 5 Hz. Ligando os pontos, enxerga-se uma oscilação mais lenta, e a linha tracejada mostra que as mesmas amostras pertencem também a um cosseno de 3 Hz. A simulação confirmou isso numericamente: a diferença máxima entre as amostras e o cosseno de 3 Hz foi de 3×10^{-15} , ou seja, zero dentro da precisão do computador.

A equação explica esse resultado. Com $f_s = 8$ Hz:

$$\cos\left(2\pi \frac{5}{8}n\right) = \cos\left(2\pi n - 2\pi \frac{3}{8}n\right) = \cos\left(2\pi \frac{3}{8}n\right)$$

Como n é inteiro, somar ou retirar $2\pi n$ não altera o cosseno. Duas senoides contínuas diferentes (5 Hz e 3 Hz) geram exatamente a mesma sequência discreta, e depois da amostragem não é mais possível saber qual delas existia.

A conclusão é que a frequência de amostragem define a fidelidade da representação discreta. Quanto maior f_s em relação à frequência do sinal, mais a sequência se parece com o sinal original. Quando f_s é baixa demais, a sequência passa a representar outro sinal. O limite exato entre as duas situações é dado pelo Teorema de Nyquist, que será estudado na Parte 2.

O custo de aumentar f_s também aparece na tabela: passar de 25 Hz para 100 Hz multiplicou por quatro o número de amostras a armazenar e processar.

4. Tópico 3 – Quantização e Resolução

4.1 Resumo conceitual

A amostragem torna o tempo discreto, mas cada amostra ainda pode ter qualquer valor real. Um sistema digital guarda cada amostra com um número finito de bits, e por isso só consegue representar um número finito de amplitudes. Quantizar é substituir cada amostra pelo nível permitido mais próximo.

Amostragem e quantização são, portanto, operações diferentes. A amostragem discretiza o eixo horizontal (tempo) e a quantização discretiza o eixo vertical (amplitude). A diferença entre o valor quantizado e o valor original é o erro de quantização, que não pode ser desfeito depois.

4.2 Formulação matemática

Um conversor de N bits, operando entre 0 e V_{ref} , possui L níveis separados por um passo ΔV , chamado resolução:

$$L = 2^N, \quad \Delta V = \frac{V_{ref}}{2^N}$$

O quantizador usado arredonda cada amostra para o nível mais próximo, e o erro é a diferença entre a saída e a entrada:

$$x_q[n] = \Delta V \cdot \text{round}\left(\frac{x[n]}{\Delta V}\right), \quad e[n] = x_q[n] - x[n]$$

Com arredondamento, a amostra nunca fica a mais de meio passo do nível escolhido. Quando o erro se distribui de forma aproximadamente uniforme dentro desse intervalo, seu valor eficaz (RMS) é conhecido:

$$|e[n]| \leq \frac{\Delta V}{2}, \quad e_{RMS} \approx \frac{\Delta V}{\sqrt{12}}$$

Cada bit adicional dobra o número de níveis e reduz a resolução e o erro pela metade.

4.3 Exemplo

O sinal escolhido é uma tensão senoidal de 1 Hz, $x(t) = 2,5 + 2,0 \sin(2\pi t)$ volts, que varia entre 0,5 V e 4,5 V. Ela foi amostrada a 100 Hz durante 1 s (100 amostras) e quantizada com $V_{ref} = 5$ V usando $N = 3, 4$ e 8 bits.

A faixa de 0,5 V a 4,5 V foi escolhida para ficar dentro dos limites do conversor, evitando saturação. Assim, todo o erro observado vem da quantização.

4.4 Simulação

Código completo (arquivo `simulacoes/quantizacao/topico3_quantizacao.py`):

```
"""Tópico 3 - Quantização de um mesmo sinal com 3, 4 e 8 bits."""
import os
import numpy as np
import matplotlib.pyplot as plt

PASTA = "figuras"
os.makedirs(PASTA, exist_ok=True)
plt.rcParams.update({"font.size": 10, "axes.grid": True, "grid.alpha": 0.3})

def quantizar(x, n_bits, v_ref):
    """Quantizador uniforme entre 0 e v_ref, com arredondamento ao nível mais próximo."""
    niveis = 2 ** n_bits
    delta = v_ref / niveis
    indice = np.clip(np.round(x / delta), 0, niveis - 1)
    return indice * delta, delta, niveis

# Tensão senoidal de 1 Hz entre 0,5 V e 4,5 V, amostrada a 100 Hz
v_ref = 5.0
fs = 100.0
n = np.arange(0, 100)
t = n / fs
x = 2.5 + 2.0 * np.sin(2 * np.pi * 1.0 * t)

fig, ax = plt.subplots(3, 2, figsize=(11, 9), sharex=True)
print(" N | níveis | resolução (V) | erro máx (V) | delta/2 (V) | erro RMS (V) | delta/raiz(12) (V)")

for linha, n_bits in enumerate([3, 4, 8]):
    x_q, delta, niveis = quantizar(x, n_bits, v_ref)
    erro = x_q - x
```

```

erro_max = np.max(np.abs(erro))
erro_rms = np.sqrt(np.mean(erro ** 2))
print(f"{n_bits:2d} | {niveis:6d} | {delta:13.5f} | {erro_max:12.5f} | {delta / 2:11.5f} | "
      f"{erro_rms:12.5f} | {delta / np.sqrt(12):.5f}")

ax[linha, 0].plot(t, x, color="tab:blue", label="original")
ax[linha, 0].step(t, x_q, where="mid", color="tab:red", label="quantizado")
ax[linha, 0].set(title=f"N = {n_bits} bits, L = {niveis} níveis,  $\Delta V = {delta * 1000:.2f}$  mV",
                ylabel="Tensão (V)")
ax[linha, 0].legend(loc="upper right", fontsize=8)

ax[linha, 1].stem(t, erro * 1000, linefmt="tab:green", markerfmt=".", basefmt=" ")
ax[linha, 1].axhline(delta * 500, color="gray", linestyle="--", label="±  $\Delta V/2$ ")
ax[linha, 1].axhline(-delta * 500, color="gray", linestyle="--")
ax[linha, 1].set(title=f"Erro de quantização, N = {n_bits} bits", ylabel="Erro (mV)")
ax[linha, 1].legend(loc="upper right", fontsize=8)

ax[2, 0].set_xlabel("Tempo t (s)")
ax[2, 1].set_xlabel("Tempo t (s)")
fig.tight_layout()
fig.savefig(os.path.join(PASTA, "t3_quantizacao.png"), dpi=150)
plt.show()

```

4.5 Resultados

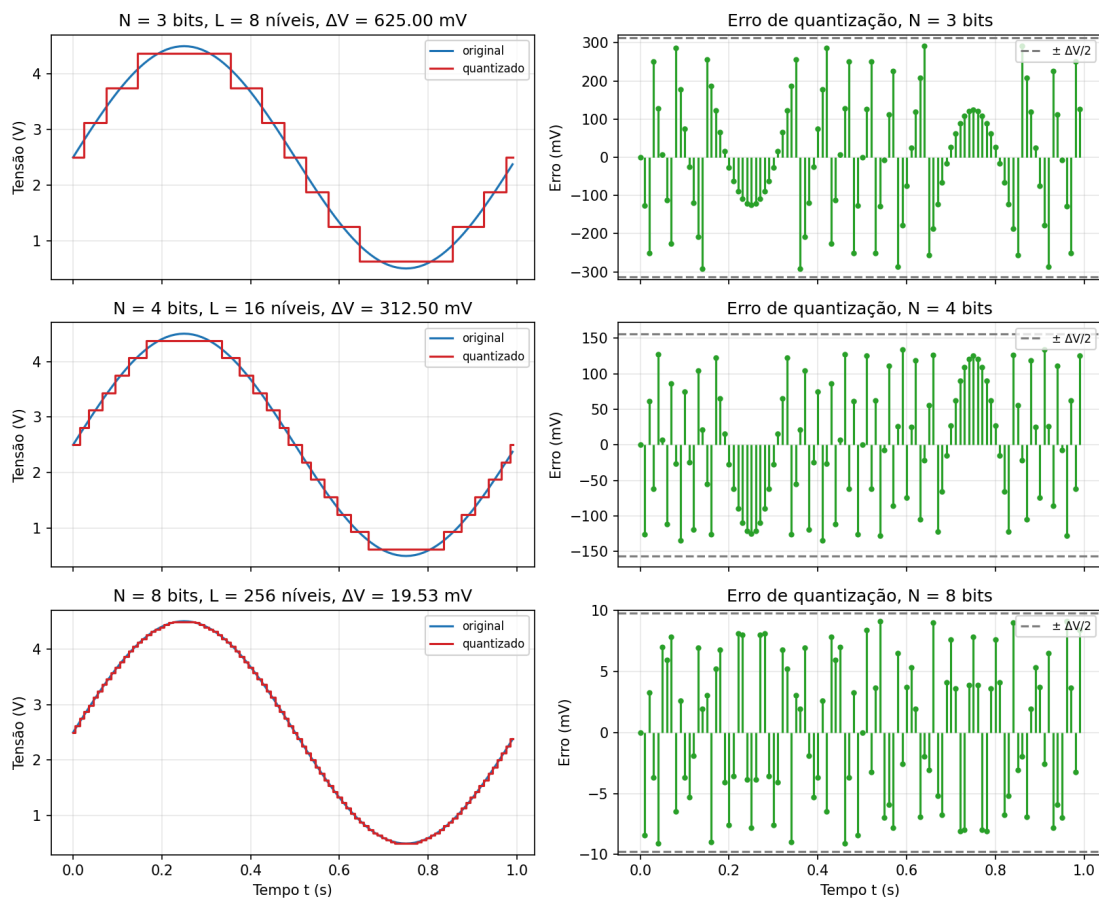


Figura 3. Sinal original e quantizado (à esquerda) e erro de quantização (à direita) para N = 3, 4 e 8 bits. As linhas tracejadas marcam $\pm \Delta V/2$.

N (bits)	Níveis L	Resolução ΔV (mV)	Erro máximo (mV)	Limite $\Delta V/2$ (mV)	Erro RMS medido (mV)	$\Delta V/\sqrt{12}$ (mV)
3	8	625,00	291,0	312,5	161,3	180,4
4	16	312,50	134,2	156,3	88,4	90,2
8	256	19,53	9,1	9,8	6,0	5,6

4.6 Discussão

Visualmente, o sinal quantizado tem forma de escada. Com 3 bits os degraus têm 625 mV e a senoide fica grosseira: perto do pico, várias amostras seguidas caem no mesmo nível de 4,375 V e o topo arredondado vira um patamar. Com 4 bits os degraus caem para 312,5 mV e a escada já acompanha melhor a curva. Com 8 bits os degraus de 19,53 mV são invisíveis na escala do gráfico, e as curvas original e quantizada se sobrepõem.

Numericamente, os resultados confirmam a formulação. O erro máximo medido ficou sempre abaixo do limite $\Delta V/2$ ($291,0 < 312,5$ mV; $134,2 < 156,3$ mV; $9,1 < 9,8$ mV), como mostram as linhas tracejadas da Figura 3. O erro RMS medido ficou próximo da previsão $\Delta V/\sqrt{12}$ nos três casos.

A relação entre bits e erro também aparece na tabela. De 3 para 4 bits, um único bit a mais dobrou os níveis (8 para 16), dividiu a resolução por 2 e reduziu o erro RMS de 161,3 mV para 88,4 mV. De 3 para 8 bits, a resolução ficou 32 vezes menor (2^5) e o erro RMS caiu 27 vezes.

A maior diferença entre o erro RMS medido e o teórico ocorre com 3 bits (161,3 contra 180,4 mV). A fórmula $\Delta V/\sqrt{12}$ supõe que o erro se espalha uniformemente entre $-\Delta V/2$ e $+\Delta V/2$. Com apenas 8 níveis e um sinal periódico, o erro não é aleatório: ele acompanha a forma do sinal, o que se vê no padrão repetitivo do gráfico de erro. Com mais bits o erro se parece cada vez mais com um ruído, e a previsão fica melhor.

A resolução é, portanto, um compromisso de projeto. Mais bits significam menor erro, mas também conversores mais caros e mais dados por amostra. Uma limitação desta simulação é considerar um conversor ideal, sem ruído próprio nem não linearidades.

5. Tópico 4 – Sequências Fundamentais

5.1 Resumo conceitual

Algumas sequências simples funcionam como blocos de construção: qualquer sinal discreto pode ser escrito a partir delas, e a resposta de um sistema a elas revela como o sistema se comporta.

- **Impulso unitário $\delta[n]$:** vale 1 em $n = 0$ e zero no resto. É o sinal mais simples possível, uma única amostra.
- **Degrau unitário $u[n]$:** vale zero antes de $n = 0$ e 1 a partir dele. Representa algo que é ligado e permanece ligado.
- **Exponencial a^n :** cada amostra é a anterior multiplicada por a . Descreve crescimento ou decaimento.

- **Senoide discreta:** descreve oscilações, com amplitude A, frequência angular ω_0 (em radianos por amostra) e fase ϕ .
- **Exponencial complexa $e^{j\omega n}$:** reúne um cosseno e um seno em um único número complexo que gira sobre o círculo unitário.

5.2 Formulação matemática

Impulso e degrau unitários:

$$\delta[n] = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases} \quad u[n] = \begin{cases} 1, & n \geq 0 \\ 0, & n < 0 \end{cases}$$

Os dois estão ligados. O impulso é a diferença entre o degrau e o degrau atrasado de uma amostra, pois os dois degraus só diferem em $n = 0$:

$$\delta[n] = u[n] - u[n - 1]$$

A propriedade mais importante do impulso é a de que qualquer sequência pode ser escrita como uma soma de impulsos deslocados e ponderados:

$$x[n] = \sum_{k=-\infty}^{+\infty} x[k] \delta[n - k]$$

O significado é o seguinte. O termo $\delta[n - k]$ é um impulso posicionado em $n = k$. Multiplicado por $x[k]$, ele vira uma única amostra com a altura que o sinal tem nessa posição. Somando uma dessas amostras para cada k , o sinal inteiro é reconstruído, amostra por amostra. Para um n fixo, todos os termos da soma se anulam, exceto o termo $k = n$, que vale $x[n]$. Essa decomposição é a base da convolução: se um sistema é linear e invariante no tempo, basta conhecer sua resposta a um impulso para conhecer a resposta a qualquer entrada.

Exponencial, senoide discreta e exponencial complexa:

$$x[n] = a^n u[n] \quad x[n] = A \cos(\omega_0 n + \phi) \quad e^{j\omega n} = \cos(\omega n) + j \sin(\omega n)$$

A última expressão é a identidade de Euler. Dela se obtém o cosseno e o seno como combinações de duas exponenciais complexas:

$$\cos(\omega n) = \frac{e^{j\omega n} + e^{-j\omega n}}{2} \quad \sin(\omega n) = \frac{e^{j\omega n} - e^{-j\omega n}}{2j}$$

5.3 Exemplos e parâmetros escolhidos

Sinal	Expressão	Parâmetros
Impulso	$\delta[n]$	n de -10 a 20
Degrau	$u[n]$	n de -10 a 20
Exponencial	$a^n u[n]$	$a = 0,8$

Sinal	Expressão	Parâmetros
Senoide	$A \cos(\omega_0 n + \varphi)$	$A = 2$; $\omega_0 = \pi/8$ rad/amostra; $\varphi = \pi/4$ rad
Exponencial complexa	$e^{j\omega n}$	$\omega = \pi/8$ rad/amostra; n de 0 a 32

Para investigar a exponencial, foram testados seis valores de a : 0,5; 0,9; 1,0; 1,2; -0,8 e -1,2.

5.4 Simulação

Código completo (arquivo `simulacoes/sinais/topico4_sequencias.py`):

```

"""Tópico 4 - Sequências fundamentais."""
import os
import numpy as np
import matplotlib.pyplot as plt

PASTA = "figuras"
os.makedirs(PASTA, exist_ok=True)
plt.rcParams.update({"font.size": 10, "axes.grid": True, "grid.alpha": 0.3})

n = np.arange(-10, 21)

# Impulso e degrau unitários
delta = (n == 0).astype(float)
u = (n >= 0).astype(float)

# Exponencial  $a^n u[n]$ . O expoente só é calculado para  $n \geq 0$ .
a = 0.8
x_exp = np.where(n >= 0, a ** np.maximum(n, 0), 0.0)

# Senoide discreta  $A \cos(\omega_0 n + \text{fase})$ , com período de 16 amostras
A, w0, fase = 2.0, np.pi / 8, np.pi / 4
x_cos = A * np.cos(w0 * n + fase)

fig, ax = plt.subplots(2, 2, figsize=(11, 7))
dados = [(delta, "Impulso unitário  $\delta[n]$ "), (u, "Degrão unitário  $u[n]$ "),
          (x_exp, "Exponencial  $(0,8)^n u[n]$ "), (x_cos, "Senoide  $2 \cos(\pi n/8 + \pi/4)$ ")]
for eixo, (y, titulo) in zip(ax.ravel(), dados):
    eixo.stem(n, y, linefmt="tab:blue", markerfmt="o", basefmt=" ")
    eixo.set(title=titulo, xlabel="Índice n (amostras)", ylabel="Amplitude")
fig.tight_layout()
fig.savefig(os.path.join(PASTA, "t4_sequencias.png"), dpi=150)
plt.show()

# Verificação de  $\delta[n] = u[n] - u[n-1]$ 
u_atrasado = (n - 1 >= 0).astype(float)
print("δ[n] = u[n] - u[n-1]:", np.array_equal(delta, u - u_atrasado))

# Verificação da propriedade  $x[n] = \text{soma de } x[k] \delta[n-k]$ 
reconstruido = np.zeros(n.size)
for k, x_k in zip(n, x_cos):
    reconstruido += x_k * (n - k == 0)
print("x[n] reconstruído com impulsos deslocados:", np.allclose(reconstruido, x_cos))

# Efeito do valor de a em  $x[n] = a^n u[n]$ 
m = np.arange(0, 16)
valores_a = [0.5, 0.9, 1.0, 1.2, -0.8, -1.2]
fig, ax = plt.subplots(2, 3, figsize=(12, 6.5))
for eixo, valor in zip(ax.ravel(), valores_a):
    y = valor ** m
    eixo.stem(m, y, linefmt="tab:purple", markerfmt="o", basefmt=" ")
    eixo.set(title=f"a = {valor:g}", xlabel="Índice n (amostras)", ylabel="a^n")
    print(f"a = {valor:5.1f} | x[5] = {y[5]:9.4f} | x[15] = {y[15]:10.4f}")

```

```

fig.tight_layout()
fig.savefig(os.path.join(PASTA, "t4_exponenciais.png"), dpi=150)
plt.show()

# Exponencial complexa  $e^{j\omega n}$  e identidade de Euler
p = np.arange(0, 33)
z = np.exp(1j * w0 * p)
print("Euler, parte real igual a cos( $\omega n$ ):", np.allclose(z.real, np.cos(w0 * p)))
print("Euler, parte imaginária igual a sen( $\omega n$ ):", np.allclose(z.imag, np.sin(w0 * p)))
cos_por_exponenciais = (np.exp(1j * w0 * p) + np.exp(-1j * w0 * p)) / 2
print("cos( $\omega n$ ) = ( $e^{j\omega n}$  +  $e^{-j\omega n}$ )/2:", np.allclose(cos_por_exponenciais.imag, 0)
      and np.allclose(cos_por_exponenciais.real, np.cos(w0 * p)))
print("Módulo de  $e^{j\omega n}$  (mínimo e máximo):", np.round(np.abs(z).min(), 6), np.round(np.abs(z).max(), 6))

fig = plt.figure(figsize=(12, 4.2))
ax1 = fig.add_subplot(1, 3, 1)
ax1.stem(p, z.real, linefmt="tab:blue", markerfmt="o", basefmt=" ")
ax1.set(title="Parte real: cos( $\pi n/8$ )", xlabel="Índice n (amostras)", ylabel="Amplitude")
ax2 = fig.add_subplot(1, 3, 2)
ax2.stem(p, z.imag, linefmt="tab:red", markerfmt="o", basefmt=" ")
ax2.set(title="Parte imaginária: sen( $\pi n/8$ )", xlabel="Índice n (amostras)", ylabel="Amplitude")
ax3 = fig.add_subplot(1, 3, 3)
circulo = np.exp(1j * np.linspace(0, 2 * np.pi, 400))
ax3.plot(circulo.real, circulo.imag, color="gray", linewidth=0.8)
ax3.plot(z.real[16], z.imag[16], "o", color="tab:green")
for i in range(0, 16, 2):
    ax3.annotate(f"n={i}", (z.real[i] * 1.13, z.imag[i] * 1.13), ha="center", va="center", fontsize=8)
ax3.set(title=" $e^{j\pi n/8}$  no plano complexo", xlabel="Parte real", ylabel="Parte imaginária",
        xlim=(-1.4, 1.4), ylim=(-1.4, 1.4))
ax3.set_aspect("equal")
fig.tight_layout()
fig.savefig(os.path.join(PASTA, "t4_exponencial_complexa.png"), dpi=150)
plt.show()

```

5.5 Resultados

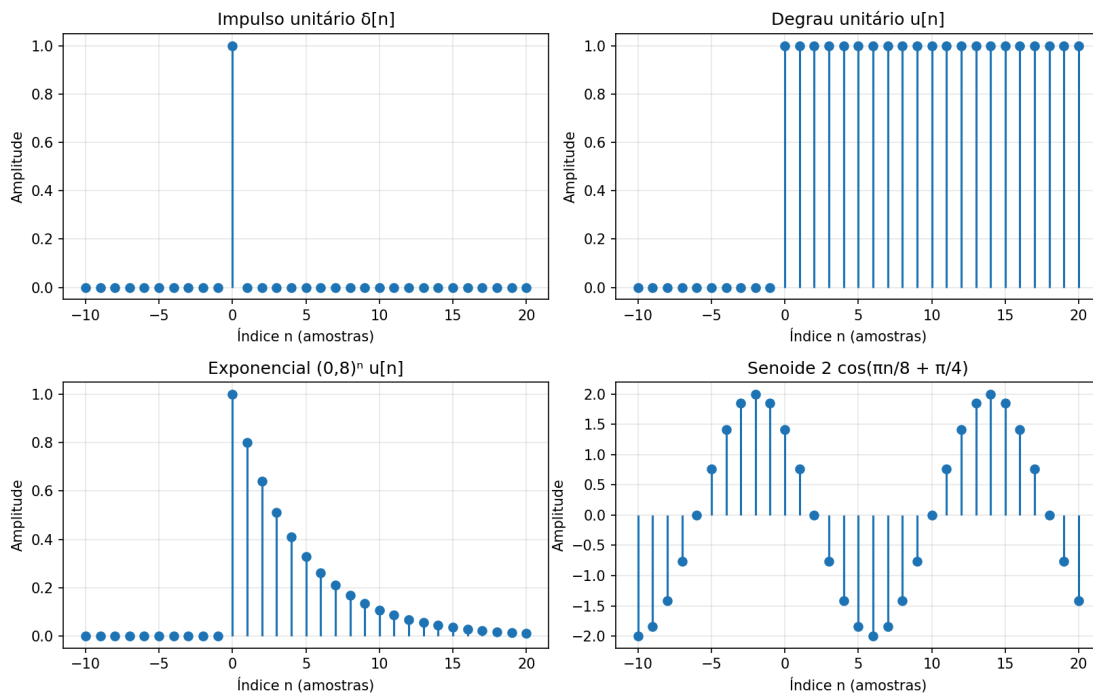


Figura 4. As quatro sequências pedidas: $\delta[n]$, $u[n]$, $(0,8)^n u[n]$ e $2 \cos(\pi n/8 + \pi/4)$.

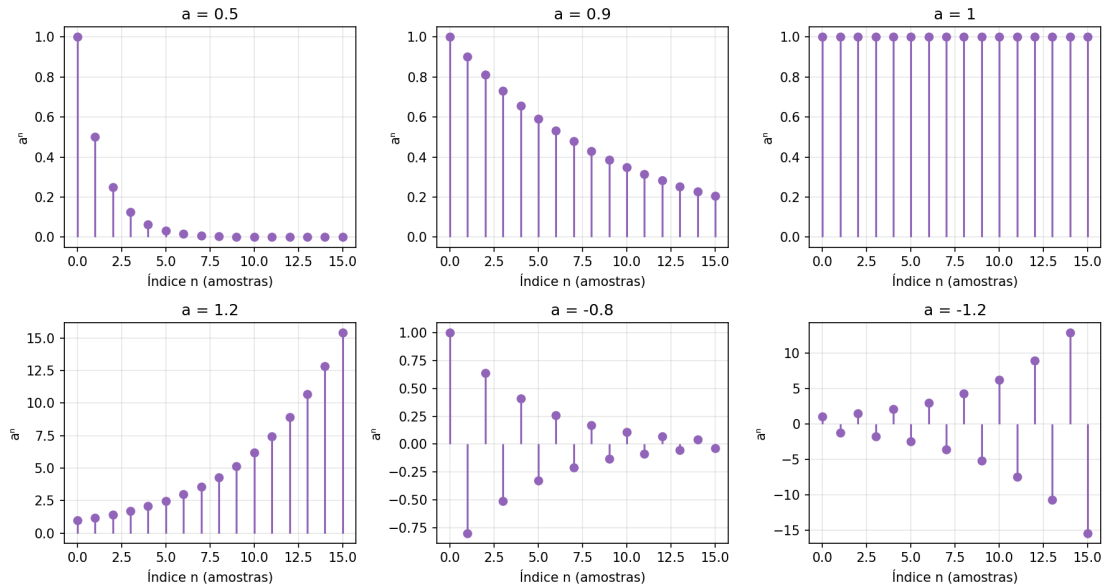


Figura 5. Sequência a^n para seis valores de a .

a	$x[5]$	$x[15]$	Comportamento
0,5	0,0312	0,00003	Decai rapidamente para zero
0,9	0,5905	0,2059	Decai lentamente para zero
1,0	1,0000	1,0000	Constante (igual ao degrau)
1,2	2,4883	15,4070	Cresce sem limite
-0,8	-0,3277	-0,0352	Decai alternando o sinal
-1,2	-2,4883	-15,4070	Cresce alternando o sinal

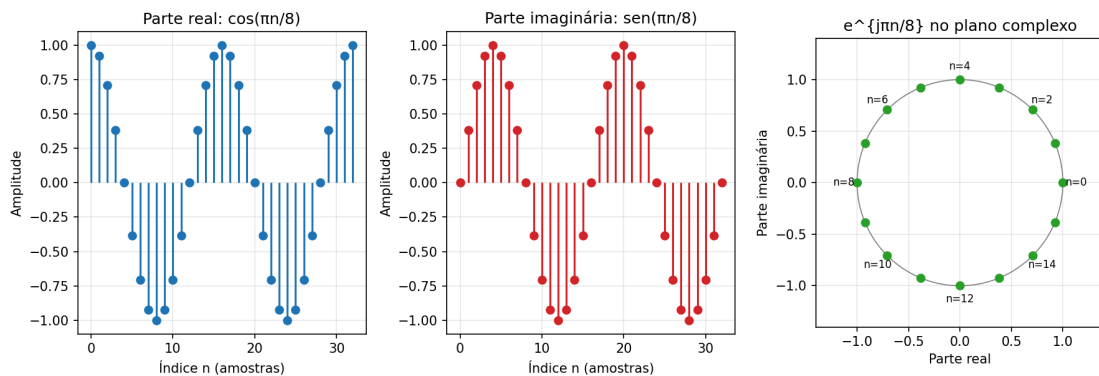


Figura 6. Exponencial complexa com $\omega = \pi/8$: parte real, parte imaginária e posição no plano complexo para n de 0 a 15.

As quatro verificações numéricas retornaram verdadeiro: $\delta[n] = u[n] - u[n - 1]$; a reconstrução da senoide pela soma de impulsos deslocados; a parte real de $e^{j\omega n}$ igual a $\cos(\omega n)$ e a imaginária igual a $\sin(\omega n)$; e $\cos(\omega n) = (e^{j\omega n} + e^{-j\omega n})/2$. O módulo de $e^{j\omega n}$ foi igual a 1 em todas as amostras.

5.6 Discussão

Impulso e degrau. Na Figura 4, o impulso é uma única haste em $n = 0$ e o degrau é uma fileira de hastes de altura 1 a partir de $n = 0$. A relação $\delta[n] = u[n] - u[n - 1]$ pode ser lida no próprio gráfico: subtraindo do degrau uma cópia dele atrasada de uma amostra, todas as hastes se cancelam, menos a de $n = 0$.

Exponencial. O parâmetro a decide tudo, e são dois aspectos independentes. O módulo de a define se a sequência decai ($|a| < 1$), fica constante ($|a| = 1$) ou cresce ($|a| > 1$). O sinal de a define se as amostras mantêm o sinal ($a > 0$) ou alternam entre positivo e negativo ($a < 0$), pois a^n é positivo para n par e negativo para n ímpar.

A tabela mostra a velocidade do decaimento: em $n = 15$, a sequência com $a = 0,5$ já vale 0,00003, enquanto a de $a = 0,9$ ainda vale 0,2059. Quanto mais próximo de 1 está $|a|$, mais longa é a “memória” da sequência. Já com $a = 1,2$ o valor passa de 15 em apenas 15 amostras. Esse comportamento será importante no estudo de estabilidade: respostas com $|a| > 1$ crescem sem limite.

Senoide. Com $\omega_0 = \pi/8$, o período é $N = 2\pi/\omega_0 = 16$ amostras, e a Figura 4 confirma que o padrão se repete a cada 16 hastes. A amplitude $A = 2$ limita os valores entre -2 e $+2$. A fase $\phi = \pi/4$ adianta a curva: em $n = 0$ o valor é $2 \cos(\pi/4) = 1,414$, e não o máximo. O máximo ocorreria quando $\omega_0 n + \phi = 0$, ou seja, em $n = -2$.

Exponencial complexa. A Figura 6 mostra as duas faces da identidade de Euler. A parte real é um cosseno e a parte imaginária é um seno, ambos com período de 16 amostras e defasados de um quarto de período (4 amostras). No plano complexo, as amostras são pontos sobre o círculo de raio 1, e cada nova amostra avança um ângulo $\omega = \pi/8$ (22,5°). Depois de 16 amostras o ponto volta ao início.

A relação entre senoides e exponenciais complexas é, portanto, geométrica: a senoide é a sombra (projeção) de um ponto que gira com velocidade constante. O cosseno é a projeção sobre o eixo real e o seno é a projeção sobre o eixo imaginário. Somando duas exponenciais que giram em sentidos opostos, as partes imaginárias se cancelam e sobra o cosseno real.

Essa relação é útil porque exponenciais são mais fáceis de manipular do que senos e cossenos: atrasar uma exponencial complexa equivale a multiplicá-la por uma constante. Por isso a Transformada de Fourier, tema da Parte 2, decompõe os sinais em exponenciais complexas.

6. Tópico 5 – Operações com Sinais

6.1 Resumo conceitual

As operações básicas sobre sinais discretos se dividem em dois grupos. As operações sobre a variável independente mexem no índice n e mudam a posição das amostras no eixo horizontal: deslocamento e inversão temporal. As operações sobre a amplitude mexem no valor das amostras e mudam o eixo vertical: escalonamento e soma.

Essas quatro operações são exatamente as usadas na convolução (inversão, deslocamento, multiplicação e soma), e por isso entendê-las bem é um pré-requisito para o restante da disciplina.

6.2 Formulação matemática

Operação	Expressão	Efeito
Atraso	$y[n] = x[n - n_0]$	Desloca o sinal n_0 amostras para a direita
Avanço	$y[n] = x[n + n_0]$	Desloca o sinal n_0 amostras para a esquerda
Inversão temporal	$y[n] = x[-n]$	Espelha o sinal em torno de $n = 0$
Escalonamento de amplitude	$y[n] = A x[n]$	Multiplica cada amostra por A
Soma	$y[n] = x_1[n] + x_2[n]$	Soma as amostras de mesmo índice

A regra para não errar o sentido do deslocamento é perguntar em que n o argumento vale zero. Em $x[n - 2]$, o argumento é zero quando $n = 2$. Logo, a amostra que estava em 0 passa a aparecer em 2: o sinal foi para a direita, ou seja, atrasou.

6.3 Exemplo

Foi escolhida uma rampa finita, de propósito assimétrica, para que a inversão e os deslocamentos fiquem visíveis:

$$x[n] = \begin{cases} n + 1, & 0 \leq n \leq 4 \\ 0, & \text{caso contrário} \end{cases} \Rightarrow x[n] = \{1, 2, 3, 4, 5\} \text{ para } n = 0, \dots, 4$$

Um sinal simétrico, como um pulso retangular centrado em zero, seria uma má escolha: $x[-n]$ ficaria idêntico a $x[n]$ e a inversão não seria percebida.

Para a soma, a rampa $x_1[n] = x[n]$ foi somada a um pulso retangular $x_2[n]$ de amplitude 2 entre $n = -2$ e $n = 2$.

6.4 Simulação

O sinal foi escrito como uma função de n . Assim, cada operação sobre o índice é feita exatamente como na expressão matemática: para obter $x[n - 2]$ basta chamar $x(n - 2)$.

Código completo (arquivo `simulacoes/sinais/topico5_operacoes.py`):

```
"""Tópico 5 - Operações com sinais discretos."""
import os
import numpy as np
import matplotlib.pyplot as plt

PASTA = "figuras"
os.makedirs(PASTA, exist_ok=True)
plt.rcParams.update({"font.size": 10, "axes.grid": True, "grid.alpha": 0.3})
```

```

def x(n):
    """Rampa finita:  $x[n] = n + 1$  para  $0 \leq n \leq 4$  e zero fora desse intervalo."""
    n = np.arange(n)
    return np.where((n >= 0) & (n <= 4), n + 1, 0).astype(float)

n = np.arange(-8, 9)
original = x(n)

operacoes = [
    ("x[n] (original)", original),
    ("x[n - 2] (atraso de 2)", x(n - 2)),
    ("x[n + 2] (avanço de 2)", x(n + 2)),
    ("x[-n] (inversão temporal)", x(-n)),
    ("2x[n] (ganho 2)", 2 * original),
    ("-x[n] (inversão de sinal)", -original),
]

fig, ax = plt.subplots(2, 3, figsize=(13, 7), sharex=True, sharey=True)
print("n      :", n)
for eixo, (titulo, y) in zip(ax.ravel(), operacoes):
    eixo.stem(n, original, linefmt="silver", markerfmt="s", basefmt=" ", label="x[n] original")
    eixo.stem(n, y, linefmt="tab:blue", markerfmt="o", basefmt=" ", label="resultado")
    eixo.set(title=titulo, ylim=(-6, 11))
    eixo.axvline(0, color="black", linewidth=0.6)
    eixo.legend(loc="upper left", fontsize=8)
    nao_nulos = n[y != 0]
    print(f"{{titulo:28s}} | não nulo de n = {{nao_nulos.min():2d}} até {{nao_nulos.max():2d}} | "
          f"valores = {{y[y != 0]}}")
for eixo in ax[1]:
    eixo.set_xlabel("Índice n (amostras)")
for eixo in ax[:, 0]:
    eixo.set_ylabel("Amplitude")
fig.tight_layout()
fig.savefig(os.path.join(PASTA, "t5_operacoes.png"), dpi=150)
plt.show()

# Soma de sinais: a rampa mais um pulso retangular de amplitude 2 entre n = -2 e n = 2
x1 = original
x2 = np.where((n >= -2) & (n <= 2), 2.0, 0.0)
y = x1 + x2
print("x1[n] + x2[n] para n = -3..5:", y[(n >= -3) & (n <= 5)])

fig, ax = plt.subplots(1, 3, figsize=(13, 3.8), sharey=True)
for eixo, (titulo, sinal, cor) in zip(ax, [("x1[n] (rampa)", x1, "tab:blue"),
                                           ("x2[n] (pulso)", x2, "tab:orange"),
                                           ("y[n] = x1[n] + x2[n]", y, "tab:green")]):
    eixo.stem(n, sinal, linefmt=cor, markerfmt="o", basefmt=" ")
    eixo.set(title=titulo, xlabel="Índice n (amostras)")
ax[0].set_ylabel("Amplitude")
fig.tight_layout()
fig.savefig(os.path.join(PASTA, "t5_soma.png"), dpi=150)
plt.show()

```

6.5 Resultados

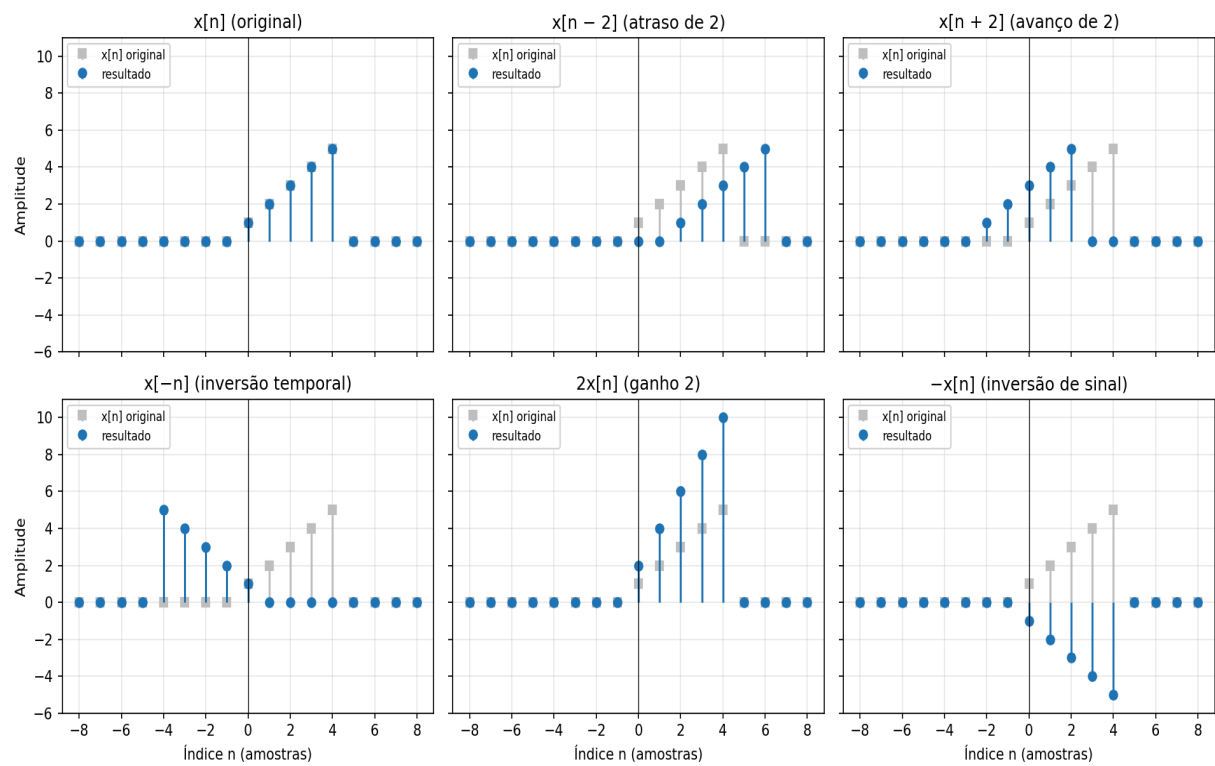


Figura 7. As seis sequências pedidas. Em cada gráfico, o sinal original aparece em cinza e o resultado em azul.

Operação	Intervalo não nulo	Valores
$x[n]$	$n = 0$ a 4	1, 2, 3, 4, 5
$x[n - 2]$	$n = 2$ a 6	1, 2, 3, 4, 5
$x[n + 2]$	$n = -2$ a 2	1, 2, 3, 4, 5
$x[-n]$	$n = -4$ a 0	5, 4, 3, 2, 1
$2x[n]$	$n = 0$ a 4	2, 4, 6, 8, 10
$-x[n]$	$n = 0$ a 4	-1, -2, -3, -4, -5

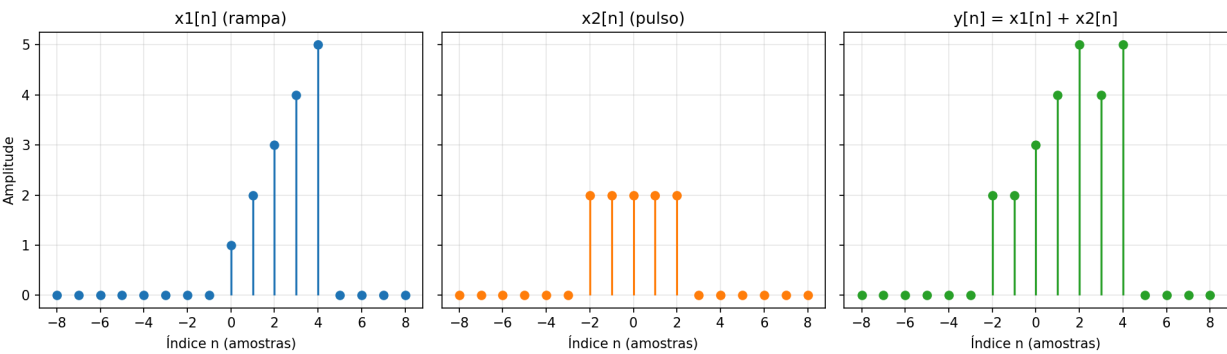


Figura 8. Soma de sinais: rampa $x1[n]$, pulso $x2[n]$ e resultado $y[n] = x1[n] + x2[n]$.

6.6 Discussão

$x[n - 2]$, atraso. A rampa manteve a forma e os valores, mas passou a ocupar $n = 2$ a 6 .

Matematicamente, $y[n] = x[n - 2]$ significa que a saída no instante n é a entrada de dois instantes atrás. Por exemplo, $y[2] = x[0] = 1$ e $y[6] = x[4] = 5$. O sinal chega “mais tarde”.

$x[n + 2]$, avanço. A rampa passou a ocupar $n = -2$ a 2 . Agora $y[n] = x[n + 2]$ é a entrada de dois instantes à frente: $y[-2] = x[0] = 1$. O sinal chega “mais cedo”. O sentido é contrário ao que o sinal de $+$ sugere à primeira vista, e o gráfico ajuda a fixar a regra.

$x[-n]$, inversão temporal. O sinal foi espelhado em torno de $n = 0$ e os valores aparecem na ordem contrária: $5, 4, 3, 2, 1$ de $n = -4$ a 0 . Como $y[n] = x[-n]$, a amostra que estava em $n = 4$ foi para $n = -4$. A única amostra que não muda de lugar é a de $n = 0$, pois $y[0] = x[0] = 1$. A rampa que subia para a direita agora desce para a direita.

$2x[n]$, escalonamento. As posições não mudaram: o sinal continua entre $n = 0$ e 4 . Cada amostra dobrou de valor, e o máximo passou de 5 para 10 . O eixo do tempo não é afetado.

$-x[n]$, inversão de sinal. É um escalonamento com $A = -1$. As posições são as mesmas e cada amostra troca de sinal. O gráfico é espelhado em torno do eixo horizontal.

A comparação entre $x[-n]$ e $-x[n]$ é a mais instrutiva, porque as duas expressões se parecem e fazem coisas completamente diferentes. O sinal de menos dentro dos colchetes atua sobre o tempo e espelha o gráfico em torno do eixo vertical. Fora dos colchetes, atua sobre a amplitude e espelha o gráfico em torno do eixo horizontal.

Soma. A soma é feita amostra por amostra. Para n de -3 a 5 , o resultado foi $y[n] = 0, 2, 2, 3, 4, 5, 4, 5, 0$. Em $n = -2$ e -1 só existe o pulso ($y = 2$). Entre $n = 0$ e 2 os dois sinais se sobrepõem ($1 + 2 = 3, 2 + 2 = 4, 3 + 2 = 5$). Em $n = 3$ e 4 só existe a rampa (4 e 5). O resultado ocupa a união dos dois intervalos, de $n = -2$ a 4 .

Todos os resultados eram esperados e coincidem com a formulação. Uma limitação prática aparece no avanço: em um sistema que opera em tempo real, $x[n + 2]$ exigiria conhecer amostras que ainda não chegaram. Na simulação isso é possível porque o sinal inteiro já está armazenado.

7. Tópico 6 – Energia e Potência

7.1 Resumo Conceitual

Os sinais em tempo discreto $x[n]$ são classificados com base em seu comportamento de acúmulo ao longo do tempo infinito ($n \in \mathbb{Z}$):

- **Sinal de Energia:** Possui energia total finita e não nula ($0 < E < \infty$). A sua potência média é nula ($P = 0$), pois a energia finita é distribuída por um intervalo de tempo que tende ao infinito. Apresenta perfil de duração finita (pulsos, janelas temporais) ou atenuação exponencial que tende a zero quando $n \rightarrow \infty$.

- **Sinal de Potência:** Possui potência média finita e não nula ($0 < P < \infty$). A sua energia total é infinita ($E = \infty$). Apresenta perfil periódico (senóides, ondas quadradas) ou ilimitado com amplitude constante ao longo do tempo (como o degrau unitário $u[n]$).
- **Sinais Sem Classificação:** Sinais que crescem ilimitadamente (ex: $2^n u[n]$) possuem $E = \infty$ e $P = \infty$, não pertencendo a nenhuma das duas categorias.

7.2 Formulação Matemática

As definições gerais de Energia Total (E) e Potência Média (P) para um sinal discreto $x[n]$ são dadas por:

- **Energia Total (E):**

$$E = \sum_{n=-\infty}^{\infty} |x[n]|^2$$

- **Potência Média (P):**

$$P = \lim_{N \rightarrow \infty} \frac{1}{2N+1} \sum_{n=-N}^N |x[n]|^2$$

Para sinais periódicos de período fundamental N_0 , a potência média simplifica-se para a média da energia acumulada em um único período:

$$P = \frac{1}{N_0} \sum_{n=0}^{N_0-1} |x[n]|^2$$

7.3 Exemplo (Cálculo Analítico)

Exemplo 1: Sinal de Energia — Pulso Exponencial Causal

Considere o sinal:

$$x_1[n] = (0,8)^n u[n]$$

Cálculo analítico da Energia (E_1):

$$E_1 = \sum_{n=-\infty}^{\infty} |(0,8)^n u[n]|^2 = \sum_{n=0}^{\infty} (0,8)^{2n} = \sum_{n=0}^{\infty} (0,64)^n$$

Utilizando a soma da série geométrica infinita $\sum_{n=0}^{\infty} r^n = \frac{1}{1-r}$ (para $|\text{r}| < 1$):

$$E_1 = \frac{1}{1-0,64} = \frac{1}{0,36} = \frac{25}{9} \approx 2,7778 J$$

(A potência média do sinal de energia é $P_1 = 0W$).

Exemplo 2: Sinal de Potência — Senóide Discreta Periódica

Considere o sinal periódico de período $N_0 = 4$:

$$x_2[n] = \cos\left(\frac{\pi}{2}n\right)$$

Cálculo analítico da Potência Média (P_2):

Avaliando as amostras ao longo de um período completo ($n = 0, 1, 2, 3$):

- $x_2[0] = \cos(0) = 1$
- $x_2[1] = \cos(\pi/2) = 0$
- $x_2[2] = \cos(\pi) = -1$
- $x_2[3] = \cos(3\pi/2) = 0$

Substituindo na fórmula para sinais periódicos:

$$P_2 = \frac{1}{4} (|1|^2 + |0|^2 + |-1|^2 + |0|^2) = \frac{1}{4}(1 + 0 + 1 + 0) = \frac{2}{4} = 0,5 W$$

(A energia total do sinal de potência é $E_2 = \infty J$).

7.4 Simulação

Abaixo está o código completo em **Python** utilizando numpy e matplotlib para realizar o cálculo numérico e plotar os gráficos exigidos:

```
import numpy as np
import matplotlib.pyplot as plt

# --- Exemplo 1: Sinal de Energia x1[n] = (0.8)^n * u[n] ---
n1 = np.arange(0, 100) # Aproximação de n=0 a infinito
x1 = (0.8)**n1

# Cálculo numérico da energia
E1_num = np.sum(np.abs(x1)**2)
print(f"Exemplo 1 - Energia Numérica: {E1_num:.4f} (Teórico: {25/9:.4f})")

# --- Exemplo 2: Sinal de Potência x2[n] = cos(pi/2 * n) ---
n2 = np.arange(-100, 101) # Intervalo de -N a N com N=100
x2 = np.cos(np.pi / 2 * n2)
```

```

# Cálculo numérico da potência média  $P = \lim (1 / (2N + 1)) * \sum(|x[n]|^2)$ 
N = 100
P2_num = (1 / (2 * N + 1)) * np.sum(np.abs(x2)**2)
print(f"Exemplo 2 - Potência Numérica: {P2_num:.4f} (Teórico: 0.5000)")

# --- Plotagem dos Gráficos ---
plt.figure(figsize=(12, 5))

# Gráfico 1: Sinal de Energia
plt.subplot(1, 2, 1)
markerline, stemlines, baseline = plt.stem(n1[:20], x1[:20])
plt.title(r'Sinal de Energia:  $x_1[n] = 0.8^n u[n]$ ')
plt.xlabel('n')
plt.ylabel('x1[n]')
plt.grid(True)

# Gráfico 2: Sinal de Potência
plt.subplot(1, 2, 2)
markerline, stemlines, baseline = plt.stem(n2[90:111], x2[90:111])
plt.title(r'Sinal de Potência:  $x_2[n] = \cos(\pi n/2)$ ')
plt.xlabel('n')
plt.ylabel('x2[n]')
plt.grid(True)

plt.tight_layout()
plt.show()

```

7.5 Resultados e Gráficos

Exemplo 1 (Sinal de Energia):

- Energia Teórica (E_1): $2,7778J$ ($\frac{25}{9}$)
- Energia Numérica ($E_{1,num}$): $2,7778J$ (para $N = 100$)
- Potência Média (P_1): $0W$

Exemplo 2 (Sinal de Potência):

- Potência Média Teórica (P_2): $0,5000W$
- Potência Média Numérica ($P_{2,num}$): $0,5000W$ (para $N = 100$)
- Energia Total (E_2): ∞J

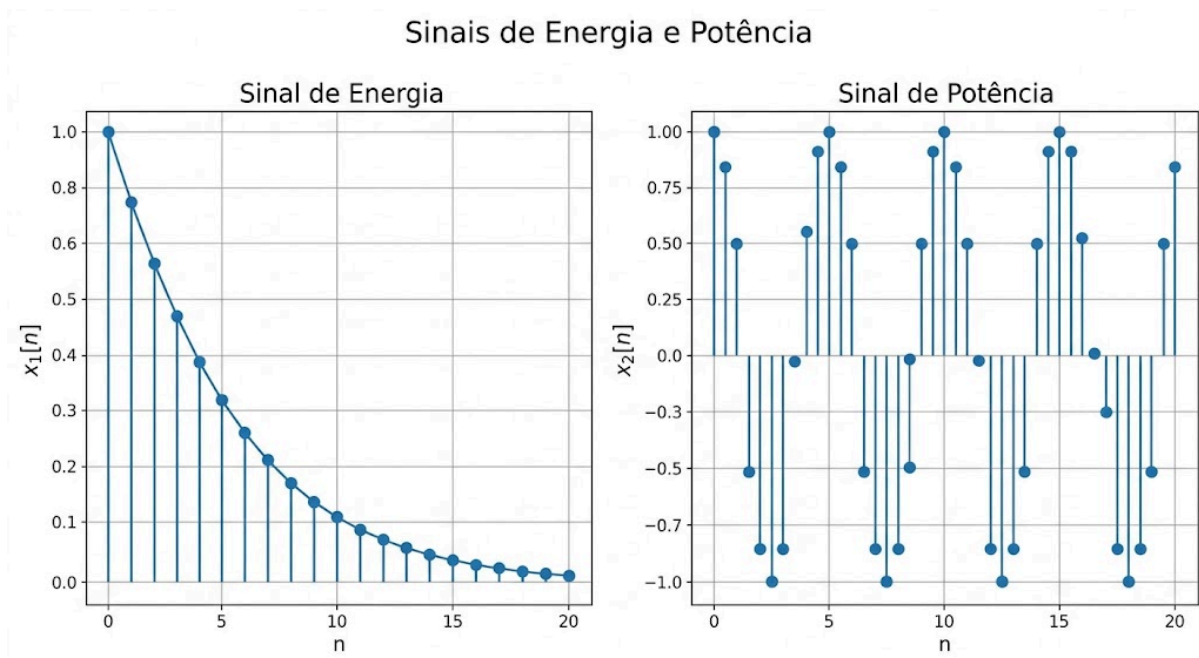


Figura 9. Sinal de energia $x_1[n]$ e sinal de potência $x_2[n]$.

7.6 Discussão dos Resultados

Concordância Analítica vs. Numérica:

- Para o sinal de energia $x_1[n]$, o valor analítico de $E_1 = 2,7778J$ converge perfeitamente com a aproximação numérica à medida que o tamanho da janela temporal aumenta, devido à rápida atenuação exponencial do termo $(0,8)^n$.
- Para o sinal de potência $x_2[n]$, a média temporal calculada no intervalo finito de $N = 100$ resulta em $P_2 = 0,5W$, coincidindo exatamente com o resultado obtido pelo cálculo analítico baseado em um único período.

Interpretação Física e Gráfica:

- O gráfico de $x_1[n]$ ilustra a natureza de um sinal transitório: a amplitude decresce rapidamente rumo a zero, garantindo que o acúmulo de energia cesse e permaneça finito.

8. Tópico 7 – Sistemas Discretos

8.1 Resumo Conceitual

Um sistema em tempo discreto é uma transformação ou operador $\mathcal{T}\{\cdot\}$ que mapeia um sinal de entrada $x[n]$ em um sinal de saída $y[n]$, denotado por:

$$y[n] = \mathcal{T}\{x[n]\}$$

A caracterização de um sistema em tempo discreto envolve a verificação de cinco propriedades fundamentais:

- **Memória:** Um sistema é **sem memória** (estático) se a saída $y[n]$ no instante presente n depende apenas da entrada $x[n]$ no mesmo instante. Se a saída depender de amostras passadas ou futuras, o sistema **possui memória** (dinâmico).
- **Causalidade:** Um sistema é **causal** se a saída no instante n depende apenas de valores presentes ($x[n]$) e/ou passados ($x[n - k]$ com $k > 0$) da entrada. Se depender de amostras futuras ($x[n + k]$ com $k > 0$), o sistema é **não causal**.
- **Linearidade:** Um sistema é **linear** se satisfaz o Princípio da Superposição (homogeneidade e aditividade). A resposta a uma combinação linear de entradas deve ser igual à mesma combinação linear das saídas individuais.
- **Invariância no Tempo:** Um sistema é **invariante no tempo** (LTI) se um deslocamento temporal na entrada produz exatamente o mesmo deslocamento temporal na saída:
 $x[n - n_0] \implies y[n - n_0]$. Se as propriedades do sistema mudarem com o tempo (ex.: coeficientes dependentes de n), ele é **variante no tempo**.
- **Estabilidade BIBO (Bounded-Input, Bounded-Output):** Um sistema é **BIBO estável** se qualquer entrada limitada ($|x[n]| \leq M_x < \infty$) produz uma saída igualmente limitada ($|y[n]| \leq M_y < \infty$).

8.2 Formulação Matemática

As definições formais para o teste de cada propriedade são dadas por:

- **Linearidade:**

$$\bullet \quad T\{a_1 x_1[n] + a_2 x_2[n]\} = a_1 T\{x_1[n]\} + a_2 T\{x_2[n]\} = a_1 y_1[n] + a_2 y_2[n]$$

- **Invariância no Tempo:**

Se $y[n] = T\{x[n]\}$, define-se a resposta à entrada deslocada $y_1[n] = T\{x[n - n_0]\}$. O sistema é invariante no tempo se:

$$\bullet \quad y_1[n] = y[n - n_0]$$

- **Estabilidade BIBO:**

Dado $|x[n]| \leq M_x < \infty, \forall n$:

$$\exists M_y < \infty \quad \text{tal que} \quad |y[n]| \leq M_y, \forall n$$

8.3 Exemplo (Demonstração Analítica dos Sistemas)

Sistema 1: $y[n] = 2x[n]$

- **Memória:** Depende apenas do instante presente n . → **Sem Memória**
- **Causalidade:** Utiliza apenas $x[n]$ (presente). → **Causal**
- **Linearidade:**

$$\mathcal{T}\{ax_1[n] + bx_2[n]\} = 2(ax_1[n] + bx_2[n]) = a(2x_1[n]) + b(2x_2[n]) = ay_1[n] + by_2[n] \rightarrow \textbf{Linear}$$

- **Invariância no Tempo:**

- Resposta à entrada deslocada: $y_1[n] = 2x[n - n_0]$

- Saída original deslocada: $y[n - n_0] = 2x[n - n_0]$

- Como $y_1[n] = y[n - n_0] \rightarrow \textbf{Invariante no Tempo}$

- **Estabilidade BIBO:**

$$\text{Se } |x[n]| \leq M_x, \text{ então } |y[n]| = 2|x[n]| \leq 2M_x < \infty \rightarrow \textbf{BIBO Estável}$$

Sistema 2: $y[n] = x[n] + x[n - 1]$

- **Memória:** Utiliza a amostra passada $x[n - 1]$. → **Com Memória**
- **Causalidade:** Utiliza o valor presente $x[n]$ e passado $x[n - 1]$ (sem amostras futuras).
→ **Causal**
- **Linearidade:**

$$\begin{aligned} \mathcal{T}\{ax_1[n] + bx_2[n]\} &= (ax_1[n] + bx_2[n]) + (ax_1[n - 1] + bx_2[n - 1]) \\ &= a(x_1[n] + x_1[n - 1]) + b(x_2[n] + x_2[n - 1]) = ay_1[n] + by_2[n] \rightarrow \textbf{Linear} \end{aligned}$$

- **Invariância no Tempo:**

- Entrada deslocada: $y_1[n] = x[n - n_0] + x[n - n_0 - 1]$

- Saída deslocada: $y[n - n_0] = x[n - n_0] + x[(n - n_0) - 1]$

- Como $y_1[n] = y[n - n_0]$. O sistema é **Invariante no Tempo**

- **Estabilidade BIBO:**

Se $|x[n]| \leq M_x$, pela desigualdade triangular:

$$|y[n]| = |x[n] + x[n - 1]| \leq |x[n]| + |x[n - 1]| \leq M_x + M_x = 2M_x < \infty \rightarrow \textbf{BIBO Estável}$$

Sistema 3: $y[n] = x^2[n]$

- **Memória:** Depende apenas de $x[n]$. → **Sem Memória**
- **Causalidade:** Depende apenas do instante presente n . → **Causal**
- **Linearidade:**

$$\mathcal{T}\{ax_1[n] + bx_2[n]\} = (ax_1[n] + bx_2[n])^2 = a^2x_1^2[n] + 2abx_1[n]x_2[n] + b^2x_2^2[n]$$

$$ay_1[n] + by_2[n] = ax_1^2[n] + bx_2^2[n]$$

Como os termos são diferentes, o sistema é **Não Linear**

- **Invariância no Tempo:**

- Entrada deslocada: $y_1[n] = x^2[n - n_0]$
- Saída deslocada: $y[n - n_0] = x^2[n - n_0]$
- Como $y_1[n] = y[n - n_0] \rightarrow$ **Invariante no Tempo**

- **Estabilidade BIBO:**

Se $|x[n]| \leq M_x$, então $|y[n]| = |x[n]|^2 \leq M_x^2 < \infty \rightarrow$ **BIBO Estável**

Sistema 4: $y[n] = x[n + 1]$

- **Memória:** Depende da amostra $x[n + 1] \neq x[n]$. $\rightarrow$ **Com Memória**
- **Causalidade:** O termo $x[n + 1]$ representa uma amostra futura da entrada. $\rightarrow$ **Não Causal**
- **Linearidade:**

$$\mathcal{T}\{ax_1[n] + bx_2[n]\} = ax_1[n + 1] + bx_2[n + 1] = ay_1[n] + by_2[n] \rightarrow \text{Linear}$$

- **Invariância no Tempo:**

- Entrada deslocada: $y_1[n] = x[n - n_0 + 1]$
- Saída deslocada: $y[n - n_0] = x[(n - n_0) + 1] = x[n - n_0 + 1]$
- Como $y_1[n] = y[n - n_0]$. O sistema é **Invariante no Tempo**.

- **Estabilidade BIBO:**

Se $|x[n]| \leq M_x$, então $|y[n]| = |x[n + 1]| \leq M_x < \infty \rightarrow$ **BIBO Estável**

Sistema 5: $y[n] = n \cdot x[n]$

- **Memória:** Depende apenas de $x[n]$. $\rightarrow$ **Sem Memória**
- **Causalidade:** Depende apenas do instante presente n . $\rightarrow$ **Causal**
- **Linearidade:**

$$\mathcal{T}\{ax_1[n] + bx_2[n]\} = n(ax_1[n] + bx_2[n]) = a(nx_1[n]) + b(nx_2[n]) = ay_1[n] + by_2[n] \rightarrow \text{Linear}$$

- **Invariância no Tempo:**

- Entrada deslocada: $y_1[n] = nx[n - n_0]$
- Saída deslocada: $y[n - n_0] = (n - n_0)x[n - n_0]$
- Como $nx[n - n_0] \neq (n - n_0)x[n - n_0] \rightarrow$ **Variante no Tempo (Não Invariante)**

- **Estabilidade BIBO:**

$$\text{Se } |x[n]| \leq M_x, \text{ temos } |y[n]| = |n| \cdot |x[n]| \leq |n| M_x.$$

Quando $n \rightarrow \infty, |y[n]| \rightarrow \infty$. Uma entrada limitada gera saída ilimitada, então o sistema é **Não BIBO Estável**

8.4 Simulação

```
import numpy as np

# Definição dos 5 sistemas
systems = {
    "1. y[n] = 2x[n]": lambda x, n: 2 * x,
    "2. y[n] = x[n] + x[n-1]": lambda x, n: x + np.roll(x, 1),
    "3. y[n] = x[n]^2": lambda x, n: x**2,
    "4. y[n] = x[n+1]": lambda x, n: np.roll(x, -1),
    "5. y[n] = n*x[n]": lambda x, n: n * x
}

# Parâmetros de teste
n = np.arange(-10, 11)
x1 = np.sin(np.pi / 4 * n)
x2 = np.cos(np.pi / 3 * n)
a, b = 2.0, 3.0
n0 = 2

print(f'{"Sistema":<25} | {"Linear?":<10} | {"Invariante no Tempo?":<20}')
print("-" * 60)

for name, sys in systems.items():
    # Teste de Linearidade: T{a*x1 + b*x2} vs a*T{x1} + b*T{x2}
    y_comb = sys(a * x1 + b * x2, n)
    y_lin = a * sys(x1, n) + b * sys(x2, n)
    is_linear = np.allclose(y_comb[2:-2], y_lin[2:-2])

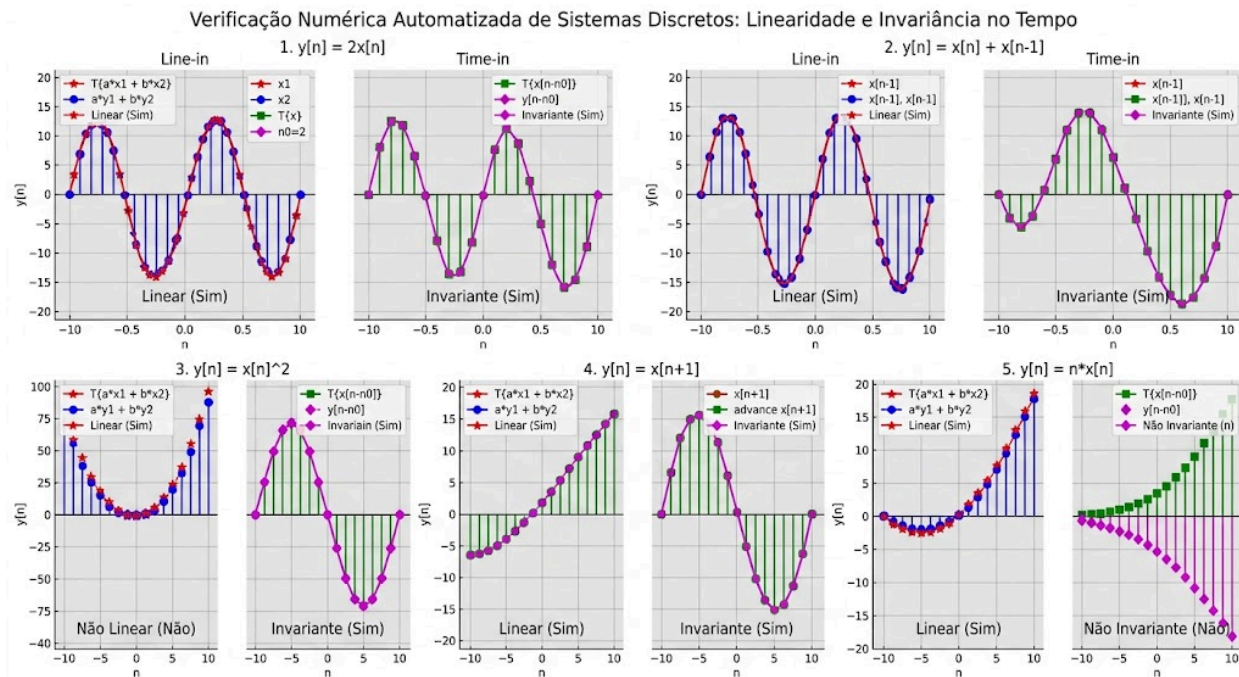
    # Teste de Invariância no Tempo: T{x[n-n0]} vs y[n-n0]
    x1_shift = np.roll(x1, n0)
    y1_shift = sys(x1_shift, n)
    y_orig_shift = np.roll(sys(x1, n), n0)
    is_time_invariant = np.allclose(y1_shift[4:-4], y_orig_shift[4:-4])

    lin_str = "Sim" if is_linear else "Não"
    ti_str = "Sim" if is_time_invariant else "Não"
```

```
print(f'{{name:<25}} | {{lin_str:<10}} | {{ti_str:<20}}')
```

8.5 Tabela geral dos resultados e gráfico

Sistema	Memória	Causalidade	Linearidade	Invariante no tempo	BIBO
$y[n] = 2x[n]$	Não	Sim	Sim	Sim	Sim
$y[n] = x[n] + x[n - 1]$	Sim	Sim	Sim	Sim	Sim
$y[n] = x^2[n]$	Não	Sim	Não	Sim	Sim
$y[n] = x[n + 1]$	Sim	Não	Sim	Sim	Sim
$y[n] = nx[n]$	Não	Sim	Sim	Não	Não



8.6 Discussão dos Resultados

A análise dos cinco sistemas demonstra que cada propriedade avalia um aspecto distinto do comportamento de um sistema discreto:

- $y[n] = 2x[n]$: Representa o caso mais simples (amplificador estático). Por não depender de amostras passadas/futuras nem do tempo n , é sem memória, causal, linear, invariante no tempo e BIBO estável.

- $y[n] = x[n] + x[n - 1]$: É um filtro de média móvel simples. A dependência do termo passado $x[n - 1]$ introduz **memória**, mas não afeta a causalidade, linearidade, invariância temporal nem a estabilidade BIBO.
- $y[n] = x^2[n]$: A operação de elevação ao quadrado viola o Princípio da Superposição, tornando o sistema **não linear**. Contudo, por operar ponto a ponto sem coeficientes temporais, ele permanece causal, sem memória, invariante no tempo e BIBO estável.
- $y[n] = x[n + 1]$: Apresenta um avanço temporal, dependendo do valor futuro $x[n + 1]$. Isso o torna **não causal** (e fisicamente irrealizável em tempo real), apesar de ser linear, invariante no tempo e BIBO estável.
- $y[n] = n \cdot x[n]$: Ilustra o impacto de um ganho variante no tempo. O fator multiplicativo n destrói a **invariância no tempo** e a **estabilidade BIBO**, pois quando $n \rightarrow \infty$, o ganho cresce ilimitadamente, fazendo com que uma entrada limitada resulte em uma saída ilimitada.

9. Tópico 8 – Sistemas LTI

9.1 Resumo Conceitual

Os sistemas Lineares e Invariantes no Tempo (LTI) constituem a classe de sistemas mais fundamental no processamento de sinais discretos. A sua análise baseia-se em conceitos-chave:

- **Linearidade:** Combina as propriedades de aditividade e homogeneidade (escalonamento). A resposta a uma combinação linear de entradas é a mesma combinação linear das respostas individuais.
- **Invariância no Tempo:** As características internas e parâmetros do sistema mantêm-se constantes no tempo. Um atraso ou avanço no sinal de entrada produz exatamente o mesmo deslocamento temporal na saída, sem alterar a sua forma.
- **Resposta ao Impulso ($h[n]$):** É a saída $y[n]$ gerada pelo sistema quando estimulado por um impulso unitário discreto $\delta[n]$ (delta de Kronecker) na origem ($n = 0$). Representa a "assinatura" dinâmica completa do sistema.
- **Decomposição em Impulsos:** Qualquer sinal discreto $x[n]$ pode ser decomposto como uma soma ponderada de impulsos unitários deslocados no tempo.
- **Caracterização Completa:** Devido à combinação da decomposição em impulsos com a linearidade e a invariância no tempo, a resposta ao impulso $h[n]$ contém toda a informação necessária para prever a saída de um sistema LTI a qualquer entrada através da **Soma de Convolução**, tratando o sistema como uma "caixa-preta".

9.2 Formulação Matemática

- **Princípio da Superposição (Linearidade):**

- $T\{ax_1[n] + bx_2[n]\} = aT\{x_1[n]\} + bT\{x_2[n]\} = ay_1[n] + by_2[n]$

- **Invariância no Tempo:**

Se $\mathcal{T}\{x[n]\} = y[n] \implies \mathcal{T}\{x[n - k]\} = y[n - k]$

- **Impulso Unitário ($\delta[n]$) e Resposta ao Impulso ($h[n]$):**

$$\delta[n] = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases} \implies h[n] = \mathcal{T}\{\delta[n]\}$$

- **Decomposição de um Sinal Arbitrário $x[n]$ em Impulsos:**

- $x[n] = \sum_{k=-\infty}^{\infty} x[k]\delta[n - k]$

- **Dedução da Soma de Convolução:**

1. Aplicação do operador do sistema à entrada decomposta:

2. $y[n] = T\{x[n]\} = T\left\{\sum_{k=-\infty}^{\infty} x[k]\delta[n - k]\right\}$

3. Uso da **Linearidade** (a transformação da soma escalada é a soma das transformações escaladas):

4. $y[n] = \sum_{k=-\infty}^{\infty} x[k]T\{\delta[n - k]\}$

5. Uso da **Invariância no Tempo** ($T\{\delta[n - k]\} = h[n - k]$):

6. $y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n - k] = x[n] * h[n]$

9.3 Exemplo (Cálculo Analítico da Convolução)

Considere um sistema LTI com resposta ao impulso dada por um pulso retangular:

$$h[n] = \delta[n] + \delta[n - 1] + \delta[n - 2]$$

Aplicando como entrada o sinal causal:

$$x[n] = 2\delta[n] + 3\delta[n - 1]$$

Cálculo da saída $y[n] = x[n] * h[n]$ via soma de convolução:

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n - k] = x[0]h[n] + x[1]h[n - 1]$$

$$y[n] = 2(\delta[n] + \delta[n - 1] + \delta[n - 2]) + 3(\delta[n - 1] + \delta[n - 2] + \delta[n - 3])$$

$$y[n] = 2\delta[n] + 5\delta[n - 1] + 5\delta[n - 2] + 3\delta[n - 3]$$

Valores das amostras de saída:

- $y[0] = 2$
- $y[1] = 5$
- $y[2] = 5$
- $y[3] = 3$
- $y[n] = 0$ para qualquer outro n .

9.4 Simulação

```
import numpy as np
import matplotlib.pyplot as plt

# Definição dos sinais finitos do exemplo
x = np.array([2, 3])      # Entrada x[n] para n = 0, 1
h = np.array([1, 1, 1])  # Resposta ao impulso h[n] para n = 0, 1, 2

# Convolução discreta y[n] = x[n] * h[n]
y = np.convolve(x, h)

# Eixos temporais
n_x = np.arange(0, len(x))
n_h = np.arange(0, len(h))
n_y = np.arange(0, len(y))

# Plotagem dos gráficos
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.stem(n_x, x)
plt.title(r'Entrada x[n]')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.grid(True)

plt.subplot(1, 3, 2)
plt.stem(n_h, h)
plt.title(r'Resposta ao Impulso h[n]')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.grid(True)
```

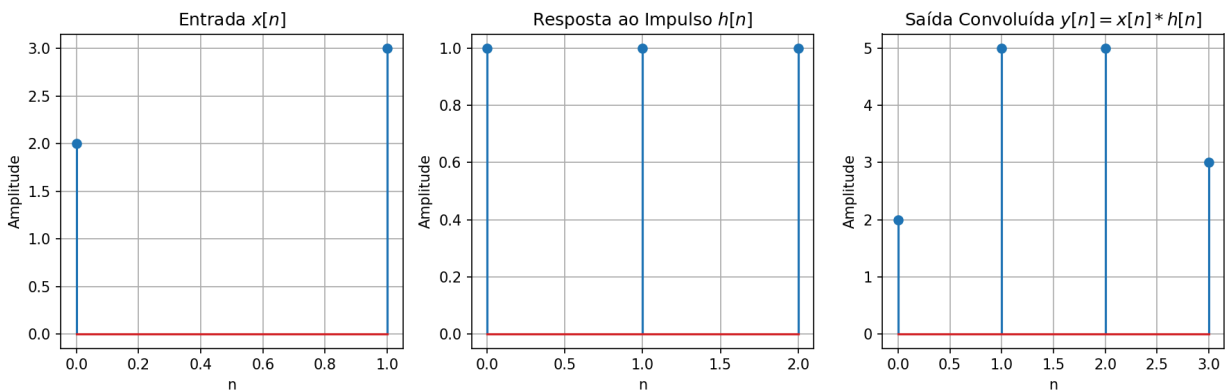
```
plt.subplot(1, 3, 3)
plt.stem(n_y, y)
plt.title(r'Saída Convoluída  $y[n] = x[n] * h[n]$ ')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.grid(True)

plt.tight_layout()
plt.show()
```

9.5 Resultados e Gráfico

- **Amostras da Entrada $x[n]$:** $[2, 3]$ para $n \in [0, 1]$
- **Amostras da Resposta ao Impulso $h[n]$:** $[1, 1, 1]$ para $n \in [0, 2]$
- **Resultado da Convolução Analítica $y[n]$:** $[2, 5, 5, 3]$ para $n \in [0, 3]$
- **Resultado da Convolução Numérica :** $[2, 5, 5, 3]$
- **Duração Temporal do Sinal de Saída:** Se $x[n]$ tem comprimento $L_x = 2$ e $h[n]$ tem comprimento $L_h = 3$, o comprimento de $y[n]$ é $L_y = L_x + L_h - 1 = 2 + 3 - 1 = 4$ amostras.

De acordo como código Python, podemos gerar os gráficos a seguir:



9.6 Discussão dos Resultados

- **Concordância dos Cálculos:** A convolução numérica obtida via script coincide exatamente com o resultado da convolução analítica obtida pela expansão algébrica dos impulsos deslocados.
- **Propriedade do Comprimento Suporte:** A saída estende-se ao longo do intervalo $n \in [0, 3]$, confirmando a regra de suporte temporal em que a duração de um sinal

convoluído é dada pela soma do tamanho dos dois sinais intervenientes menos um ($L_y = L_x + L_h - 1$).

- **Significado Físico da Operação:** A convolução funciona como um "suavizador" ou filtro: cada amostra do sinal de entrada $x[k]$ dispara uma réplica da resposta ao impulso $h[n]$, deslocada no tempo e multiplicada pela magnitude $x[k]$. A saída final $y[n]$ é o somatório de todas estas réplicas sobrepostas, provando que o conhecimento exclusivo de $h[n]$ basta para prever detalhadamente qualquer saída de um sistema LTI.

10. Tópico 9: Convolução Discreta

10.1 Resumo Conceitual

A saída $y[n]$ de um sistema Linear e Invariante no Tempo (LTI) para qualquer sinal de entrada $x[n]$ é determinada através da **Convolução Discreta** ($y[n] = x[n] * h[n]$), utilizando a resposta ao impulso $h[n]$ do sistema.

A operação de convolução é fundamentada em quatro etapas sequenciais aplicadas sobre a variável auxiliar k :

1. **Inversão (Reflexão):** Inverter a resposta ao impulso em relação à origem, obtendo $h[-k]$.
2. **Deslocamento:** Atrasar/avançar $h[-k]$ por n amostras, resultando na sequência $h[n - k]$.
3. **Multiplicação:** Multiplicar amostra por amostra o sinal de entrada $x[k]$ pela sequência invertida e deslocada $h[n - k]$.
4. **Soma:** Somar todos os produtos resultantes para determinar o valor final da amostra de saída $y[n]$.

A resposta ao impulso $h[n]$ define a natureza funcional do sistema LTI. Alterar $h[n]$ modifica a filtragem aplicada ao sinal (comportamento passa-baixas/suavizador, passa-altas/derivador ou passa-faixa), o ganho em amplitude e a duração temporal da saída.

10.2 Formulação Matemática

- **Equação Fundamental da Convolução Discreta:**

$$y[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k]h[n - k]$$

- **Comprimento Temporal do Sinal de Saída (N_y):**

Se o sinal de entrada $x[n]$ possui comprimento N_x e a resposta ao impulso $h[n]$ possui comprimento N_h , o número total de amostras não nulas da saída $y[n]$ é dado por:

- $N_y = N_x + N_h - 1$

10.3 Exemplo (Cálculo Manual Ponto a Ponto)

Experimento 1: Exemplo Orientador

Considere os sinais com início em $n = 0$:

$$x[n] = \{1, 2, 1\} (N_x = 3, \text{para } n = 0, 1, 2)$$

$$h_1[n] = \{1, 1\} (N_h = 2, \text{para } n = 0, 1)$$

O comprimento da saída será $N_y = 3 + 2 - 1 = 4$ amostras ($n \in [0, 3]$).

- **Para $n = 0$:**

- $y_1[0] = \sum_k x[k]h_1[0 - k] = x[0]h_1[0] = 1 \cdot 1 = 1$

- **Para $n = 1$:**

- $y_1[1] = \sum_k x[k]h_1[1 - k] = x[0]h_1[1] + x[1]h_1[0] = (1 \cdot 1) + (2 \cdot 1) = 1 + 2 = 3$

- **Para $n = 2$:**

- $y_1[2] = \sum_k x[k]h_1[2 - k] = x[1]h_1[1] + x[2]h_1[0] = (2 \cdot 1) + (1 \cdot 1) = 2 + 1 = 3$

- **Para $n = 3$:**

- $y_1[3] = \sum_k x[k]h_1[3 - k] = x[2]h_1[1] = 1 \cdot 1 = 1$

$$y_1[n] = \{1, 3, 3, 1\} \text{ para } n = 0, 1, 2, 3$$

Experimento 2: Modificação do Sistema ($h_2[n]$ — Filtro Derivador/Diferença)

Mantendo a mesma entrada $x[n] = \{1, 2, 1\}$ ($N_x = 3$), modifica-se a resposta ao impulso para $h_2[n] = \{1, -1\}$ ($N_h = 2$, para $n = 0, 1$).

Comprimento da saída: $N_y = 3 + 2 - 1 = 4$ amostras ($n \in [0, 3]$).

- **Para $n = 0$:** $y_2[0] = x[0]h_2[0] = 1 \cdot 1 = 1$
- **Para $n = 1$:** $y_2[1] = x[0]h_2[1] + x[1]h_2[0] = (1 \cdot (-1)) + (2 \cdot 1) = -1 + 2 = 1$
- **Para $n = 2$:** $y_2[2] = x[1]h_2[1] + x[2]h_2[0] = (2 \cdot (-1)) + (1 \cdot 1) = -2 + 1 = -1$
- **Para $n = 3$:** $y_2[3] = x[2]h_2[1] = 1 \cdot (-1) = -1$

$$y_2[n] = \{1, 1, -1, -1\} \text{ para } n = 0, 1, 2, 3$$

10.4 Simulação (Python / Octave)

Abaixo está o código em Python:

```
import numpy as np
import matplotlib.pyplot as plt

# --- Definição dos Sinais de Entrada e Respostas ao Impulso ---
x = np.array([1, 2, 1])
h1 = np.array([1, 1])    # Experimento 1 (Orientador)
h2 = np.array([1, -1])   # Experimento 2 (Modificado)

# --- Cálculo Computacional da Convolução ---
y1 = np.convolve(x, h1)
y2 = np.convolve(x, h2)

print("Resultado do Experimento 1 (Python):", y1)
print("Resultado do Experimento 2 (Python):", y2)

# --- Eixos Temporais ---
n_x = np.arange(len(x))
n_h1 = np.arange(len(h1))
n_h2 = np.arange(len(h2))
n_y1 = np.arange(len(y1))
n_y2 = np.arange(len(y2))

# --- Plotagem dos Sinais ---
fig, axes = plt.subplots(3, 2, figsize=(10, 7.5))

# Coluna 1: Experimento 1 (h1[n] = {1, 1})
axes[0, 0].stem(n_x, x)
axes[0, 0].set_title("Entrada x[n] = {1, 2, 1}")
axes[0, 0].set_ylabel("Amplitude")
```

```

axes[1, 0].stem(n_h1, h1, linefmt='r-', markerfmt='ro')
axes[1, 0].set_title("Resposta ao Impulso h1[n] = {1, 1}")
axes[1, 0].set_ylabel("Amplitude")

axes[2, 0].stem(n_y1, y1, linefmt='g-', markerfmt='go')
axes[2, 0].set_title("Saída y1[n] = x[n] * h1[n]")
axes[2, 0].set_xlabel("n")
axes[2, 0].set_ylabel("Amplitude")

# Coluna 2: Experimento 2 (h2[n] = {1, -1})
axes[0, 1].stem(n_x, x)
axes[0, 1].set_title("Entrada x[n] = {1, 2, 1}")

axes[1, 1].stem(n_h2, h2, linefmt='r-', markerfmt='ro')
axes[1, 1].set_title("Resposta ao Impulso h2[n] = {1, -1}")

axes[2, 1].stem(n_y2, y2, linefmt='g-', markerfmt='go')
axes[2, 1].set_title("Saída y2[n] = x[n] * h2[n]")
axes[2, 1].set_xlabel("n")

for ax in axes.flat:
    ax.grid(True, linestyle='--', alpha=0.6)

plt.tight_layout()
plt.show()

```

10.5 Resultados

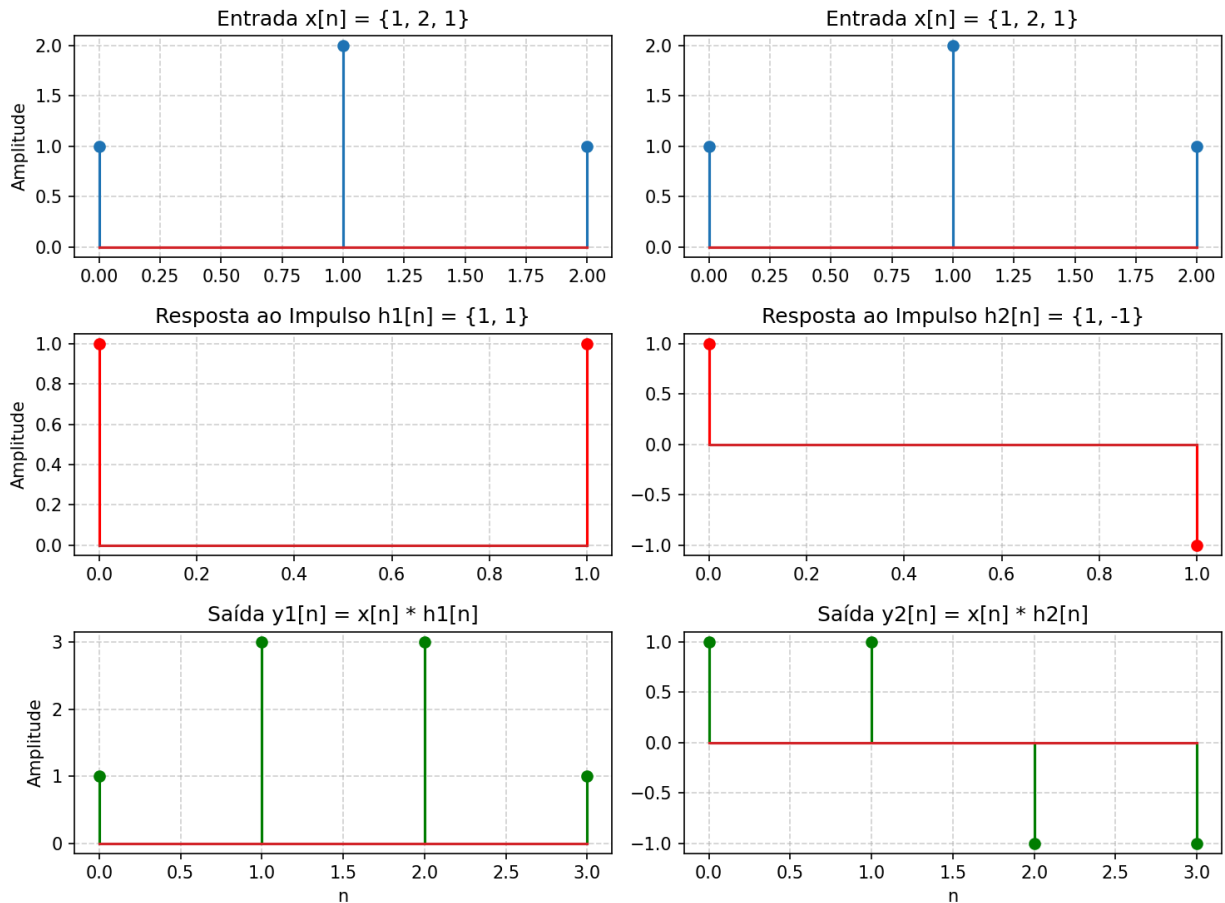
Tabela Comparativa de Verificação Numérica

Amostra (n)	Cálculo Manual y1[n]	Python numpy.convolve (y1)	Cálculo Manual y2[n]	Python numpy.convolve (y2)
0	1	1	1	1
1	3	3	1	1

2	3	3	- 1	- 1
3	1	1	- 1	- 1
Comprimento (N_y)	4 amostras	4 amostras	4 amostras	4 amostras

Conclusão da Comparação: Os resultados dos cálculos manuais efetuados ponto a ponto conferem com 100% de exatidão em relação aos valores obtidos via simulação computacional em Python.

Gráfico gerado pelo código Python.



10.6 Discussão: Como a Alteração de $h[n]$ Modifica a Saída

A resposta ao impulso $h[n]$ determina a transferência de energia e a filtragem exercida pelo sistema LTI sobre a entrada $x[n]$:

1. Atuação do Filtro Acumulador / Passa-Baixas ($h_1[n] = \{1, 1\}$):

- **Mecanismo:** Realiza a soma de amostras adjacentes. Como a soma dos coeficientes é maior que 1 ($\sum h_1 = 2$), o sistema introduz ganho de amplitude.
- **Efeito na Saída:** A saída $y_1[n] = \{1, 3, 3, 1\}$ suaviza as variações do sinal e eleva os valores de pico, atuando como um integrador/passa-baixas discreto.

2. Atuação do Filtro Derivador / Passa-Altas ($h_2[n] = \{1, -1\}$):

- **Mecanismo:** Calcula a diferença entre amostras consecutivas ($x[n] - x[n - 1]$). A soma dos seus coeficientes é nula ($\sum h_2 = 0$), o que significa que componentes contínuas (CC) são totalmente eliminadas.
- **Efeito na Saída:** A saída $y_2[n] = \{1, 1, -1, -1\}$ apresenta valores positivos nas subidas do sinal e valores negativos nas descidas, realçando transições e variações rápidas (comportamento derivador/passa-altas).

3. Garantia da Duração Temporal:

Ambos os sistemas mantêm o comprimento de saída

$N_y = N_x + N_h - 1 = 3 + 2 - 1 = 4$ amostras, confirmando que a extensão temporal da resposta depende diretamente da soma dos suportes das duas sequências envolvidas na convolução.

11. Tópico 10: Convolução como Filtragem

11.1 Resumo Conceitual

A convolução discreta é a operação matemática que viabiliza a filtragem digital em sistemas de Resposta ao Impulso Finita (FIR).

O **Filtro de Média Móvel Causal** de ordem M é um exemplo clássico de filtro FIR passa-baixas. Sua função é suavizar variações bruscas em um sinal de entrada, reduzindo o ruído aditivo de alta frequência ao calcular a média aritmética das M amostras mais recentes.

O parâmetro M (tamanho da janela do filtro) determina diretamente o desempenho e o comportamento do sistema:

- **Filtros de Janela Pequena (ex: $M = 3$):** Oferecem baixa atenuação de ruído, porém preservam a localização temporal exata e a nitidez das bordas/transições rápidas.
- **Filtros de Janela Grande (ex: $M = 10$):** Oferecem alta atenuação de ruído, produzindo um sinal bastante suave, contudo introduzem maior atraso de grupo (atraso temporal) e causam o "borramento" (suavização excessiva) de degraus e transições bruscas.

11.2 Formulação Matemática

- **Equação de Diferenças no Domínio do Tempo:**

$$y[n] = \frac{1}{M} \sum_{k=0}^{M-1} x[n - k]$$

- **Resposta ao Impulso $h[n]$ (Entrada $x[n] = \delta[n]$):**

$$h[n] = \begin{cases} \frac{1}{M}, & 0 \leq n \leq M-1 \\ 0, & \text{caso contrário} \end{cases}$$

- **Convolução Causal:**

$$y[n] = x[n] * h[n] = \sum_{k=0}^{M-1} h[k]x[n - k] = \frac{1}{M} \sum_{k=0}^{M-1} x[n - k]$$

- **Redução da Variância do Ruído Branco ($\sigma_{\text{ruído}}^2$):**

$$\sigma_{\text{saída}}^2 = \frac{\sigma_{\text{ruído}}^2}{M}$$

- **Desvio Padrão do Ruído Residual:**

$$\sigma_{\text{saída}} = \frac{\sigma_{\text{ruído}}}{\sqrt{M}}$$

- **Atraso de Grupo / Atraso Temporal (τ):**

$$\tau = \frac{M-1}{2} \text{ amostras}$$

11.3 Exemplo

Para os ordens de filtro $M = 3$, $M = 5$ e $M = 10$, as respostas ao impulso causais $h_M[n]$ são dadas por:

- **Para $M = 3$:**

- $h_3[n] = \left\{ \frac{1}{3}, \frac{1}{3}, \frac{1}{3} \right\} \approx \{0,333; 0,333; 0,333\}$ para $n = 0, 1, 2$
- Atraso temporal: $\tau = \frac{3-1}{2} = 1$ amostra
- Redução do desvio padrão do ruído: $\frac{1}{\sqrt{3}} \approx 0,577$ (mantém aproximadamente 57,7 do ruído)

- **Para $M = 5$:**

$$h_5[n] = \left\{ \frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5} \right\} = \{0,200; 0,200; 0,200; 0,200; 0,200\} \quad \text{para } n = 0, 1, \dots, 4$$

- Atraso temporal: $\tau = \frac{5-1}{2} = 2$ amostras
- Redução do desvio padrão do ruído: $\frac{1}{\sqrt{5}} \approx 0,447$ (mantém aproximadamente 44,7 % do ruído)

- **Para $M = 10$:**

$$h_{10}[n] = \left\{ \frac{1}{10}, \frac{1}{10}, \dots, \frac{1}{10} \right\} = \{0,100; 0,100; \dots; 0,100\} \quad \text{para } n = 0, 1, \dots, 9$$

- Atraso temporal: $\tau = \frac{10-1}{2} = 4,5$ amostras
- Redução do desvio padrão do ruído: $\frac{1}{\sqrt{10}} \approx 0,316$ (mantém aproximadamente 31,6% do ruído)

11.4 Simulação

O código abaixo gera o sinal sintético com pulso e degrau contaminados por ruído gaussiano ($\sigma = 0,15$), aplica os filtros de média móvel para $M = 3, 5, 10$ via convolução e plotagens correspondentes:

```
import numpy as np
import matplotlib.pyplot as plt

# Semear gerador para reprodutibilidade
np.random.seed(42)

# 1. Vetor de tempo discreto
n = np.arange(0, 80)
```

```

# 2. Sinal Original Limpo (Pulso e Degrau)
x_limpo = np.zeros_like(n, dtype=float)
x_limpo[15:35] = 1.0 # Pulso
x_limpo[50:70] = 0.6 # Degrau

# 3. Sinal Contaminado (Ruído Aditivo Gaussiano, sigma = 0.15)
ruído = np.random.normal(loc=0.0, scale=0.15, size=len(n))
x_ruidoso = x_limpo + ruído

# 4. Filtros de Média Móvel (M = 3, M = 5, M = 10)
valores_M = [3, 5, 10]
sinais_filtrados = {}
h_filtros = {}

for M in valores_M:
    h = np.ones(M) / M
    h_filtros[M] = h
    # Convolução causal truncada no comprimento de n
    sinais_filtrados[M] = np.convolve(x_ruidoso, h, mode='full')[:len(n)]

# --- Visualização 1: Filtragem do Sinal ---
plt.figure(figsize=(14, 9))

# Subplot 1: Sinais de Entrada
plt.subplot(4, 1, 1)
plt.plot(n, x_limpo, 'k--', linewidth=2, label='Sinal Original  $x[n]$ ')
plt.plot(n, x_ruidoso, color='gray', alpha=0.7, label='Sinal Contaminado  $x_{\text{ruidoso}}[n]$ ')
plt.title('Sinais de Entrada: Original (Ideal) vs. Contaminado com Ruído Gaussiano')
plt.ylabel('Amplitude')
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(loc='upper right')

# Subplots 2, 3 e 4: Resultados para M = 3, 5, 10
cores = ['blue', 'green', 'red']
for idx, M in enumerate(valores_M):
    plt.subplot(4, 1, idx + 2)
    plt.plot(n, x_limpo, 'k--', alpha=0.5, label='Original Limpo')
    plt.plot(n, sinais_filtrados[M], color=cores[idx], linewidth=2, label=f'Filtrado ( $M = {M}$ )')
    plt.title(f'Sinal Filtrado com Média Móvel  $M = {M}$ ')
    plt.ylabel('Amplitude')
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.legend(loc='upper right')

plt.xlabel('Amostras ( $n$ )')
plt.tight_layout()
plt.show()

# --- Visualização 2: Respostas ao Impulso  $h[n]$  ---
plt.figure(figsize=(12, 3))

```

```

for idx, M in enumerate(valores_M):
    plt.subplot(1, 3, idx + 1)
    n_h = np.arange(M)
    plt.stem(n_h, h_filtros[M])
    plt.title(f'Resposta ao Impulso  $h[n]$  ( $M = {M}$ )')
    plt.xlabel('n')
    plt.ylabel('h[n]')
    plt.ylim(0, 0.4)
    plt.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()

```

11.5 Resultados

O código acima gerou o seguinte gráfico:

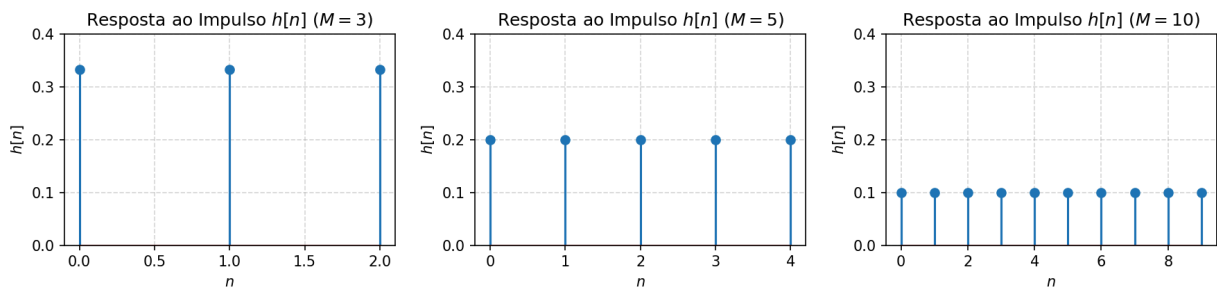


Tabela Comparativa de Desempenho dos Filtros

Métrica / Característica	M=3	M=5	M=10
Ruído Mantido ($\sigma_{saída}/\sigma_{ruído}$)	Baixa atenuação (aproximadamente 57,7%)	Média atenuação (aproximadamente 44,7%)	Alta atenuação (aproximadamente 31,6%)
Atraso Temporal ($\tau = \frac{M-1}{2}$)	1,0 amostra	2,0 amostras	4,5 amostras

Largura da Rampa de Transição	3 amostras	5 amostras	10 amostras
Nitidez de Degraus / Bordas	Preserva a forma com pequeno arredondamento	Transições moderadamente suaves	Transições fortemente borradas

11.6 Discussão dos Resultados

A análise dos resultados evidencia o compromisso fundamental (*trade-off*) no projeto de filtros digitais FIR de média móvel:

1. **Atenuação de Ruído e Suavização:** A variância do ruído gaussiano aditivo diminui inversamente com o tamanho da janela ($\sigma_{saída}^2 = \sigma_{ruído}^2 / M$). À medida que M aumenta de 3 para 10, a resposta torna-se visivelmente mais suave e livre de oscilações de alta frequência, pois a média engloba um número maior de amostras aleatórias que se cancelam mutuamente.
2. **Distorção Temporal e Degradação de Transições:**
 - **Atraso de Grupo:** Sendo um filtro causal, a resposta sofre um deslocamento temporal constante de $\tau = (M-1)/2$ amostras. Para M=10, a saída é atrasada em 4,5 amostras.
 - **Borramento de Bordas:** A transição instantânea de um degrau transforma-se em uma rampa com largura de M amostras. Janelas maiores reduzem a capacidade do sistema de acompanhar variações rápidas do sinal.
 - **Atenuação de Pulsos Curtos:** Se o sinal contiver pulsos estreitos (com largura menor que M), a média móvel atenuará severamente o pico do sinal desejado.
- **Recomendação Prática:**
 - **M pequeno (M=3):** Indicado para aplicações onde a precisão temporal e a preservação de bordas e transições rápidas são críticas, tolera-se um nível maior de ruído residual.
 - **M grande (M=10):** Indicado quando o objetivo primário é a estimação de tendências de baixa frequência ou do nível médio (CC), onde atrasos e suavização de bordas não afetam o desempenho do sistema.

12. Mini Projeto Integrador – Vibração de um Motor Elétrico

O mini projeto simula a cadeia completa de aquisição e processamento da vibração de um motor: o sinal é modelado, gerado, amostrado, quantizado, contaminado por ruído e filtrado por um sistema LTI. O filtro de média móvel escolhido ($M = 7$) reduziu o erro em relação ao sinal original de 0,301 g para 0,141 g.

Cada etapa aplica um tópico do relatório. A modelagem e a geração do sinal usam os Tópicos 1 e 4, a amostragem usa o Tópico 2 e a quantização usa o Tópico 3. O processamento final usa os sistemas LTI, a convolução e o filtro de média móvel dos Tópicos 8, 9 e 10.

O código completo está no arquivo `mini-projeto/codigo/mini_projeto.py`. Os trechos principais aparecem em cada etapa e o código completo está no Apêndice A.

12.1 Escolha do fenômeno

O fenômeno escolhido é a vibração da carcaça de um motor elétrico de indução, medida por um acelerômetro, em unidades de g ($1\text{ g} = 9,81\text{ m/s}^2$). A análise de vibração é uma técnica comum de manutenção preditiva na indústria: cada defeito mecânico ou elétrico gera vibração em uma frequência característica.

A escolha permite um modelo matemático simples (uma soma de senóides) com significado físico direto para cada parâmetro.

12.2 Etapa 1 – Modelagem do sinal

O modelo é a soma de duas componentes senoidais:

$$x(t) = A_1 \cos(2\pi f_1 t) + A_2 \cos(2\pi f_2 t) = 1,0 \cos(2\pi \cdot 30 t) + 0,4 \cos(2\pi \cdot 120 t) \quad [\text{g}]$$

Parâmetro	Valor	Significado físico
f1	30 Hz	Frequência de rotação do eixo: 1800 rpm ÷ 60 = 30 voltas por segundo
A1	1,0 g	Intensidade da vibração na frequência de rotação, típica de desbalanceamento do rotor
f2	120 Hz	Duas vezes a frequência da rede elétrica ($2 \times 60\text{ Hz}$), vibração de origem eletromagnética
A2	0,4 g	Intensidade da vibração eletromagnética, menor que a mecânica
Fases	0 rad	As duas componentes começam no máximo em $t = 0$

O maior valor do sinal é $A_1 + A_2 = 1,4\text{ g}$, em $t = 0$. A potência média teórica é $(A_1^2 + A_2^2)/2 = (1 + 0,16)/2 = 0,58\text{ g}^2$, o que corresponde a um valor eficaz de 0,76 g.

12.3 Etapa 2 – Geração computacional

O sinal foi gerado em Python durante 1 s, em uma grade fina de 100 kHz que faz o papel do sinal contínuo. Os gráficos mostram os primeiros 0,1 s (três períodos da componente de 30 Hz) para que a forma de onda fique legível.

```
import numpy as np
import matplotlib.pyplot as plt

# Etapa 1 - Modelo: vibração de um motor a 1800 rpm, medida por um acelerômetro (em g)
A1, F1 = 1.0, 30.0
A2, F2 = 0.4, 120.0

def vibracao(t):
    """Componente de rotação (30 Hz) mais componente de 120 Hz."""
    return A1 * np.cos(2 * np.pi * F1 * t) + A2 * np.cos(2 * np.pi * F2 * t)

# Etapa 2 - Geração do sinal "contínuo" em uma grade fina de 100 kHz
DURACAO = 1.0
t = np.arange(0, DURACAO, 1 / 100_000)
x_t = vibracao(t)
```

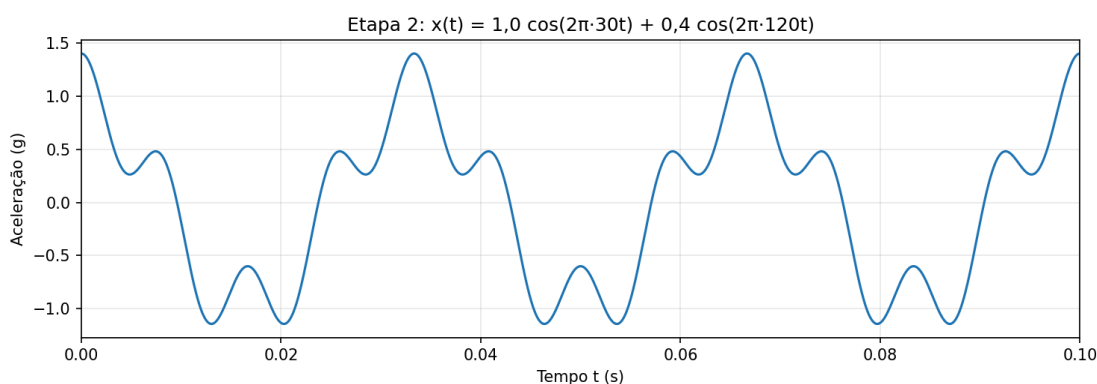


Figura 15. Sinal de vibração $x(t)$ nos primeiros 0,1 s.

A forma de onda se repete a cada 33,3 ms, o período da componente de 30 Hz. Dentro de cada período aparecem quatro ondulações menores, pois 120 Hz é exatamente quatro vezes 30 Hz. A componente de 30 Hz define a forma geral e a de 120 Hz cria os “dentes” sobre ela.

12.4 Etapa 3 – Amostragem

A frequência de amostragem escolhida foi $f_s = 2000$ Hz, o que resulta em:

$$T_s = \frac{1}{f_s} = \frac{1}{2000} = 0,5 \text{ ms}, \quad x[n] = x(nT_s)$$

Grandeza	Valor
Frequência de amostragem f_s	2000 Hz
Período de amostragem T_s	0,5 ms
Amostras em 1 s	2000

Grandeza	Valor
Amostras por período da componente de 30 Hz	66,7
Amostras por período da componente de 120 Hz	16,7

```
# Etapa 3 - Amostragem
fs = 2000.0
Ts = 1 / fs
n = np.arange(0, int(DURACAO * fs))
t_n = n * Ts
x = vibracao(t_n)
```

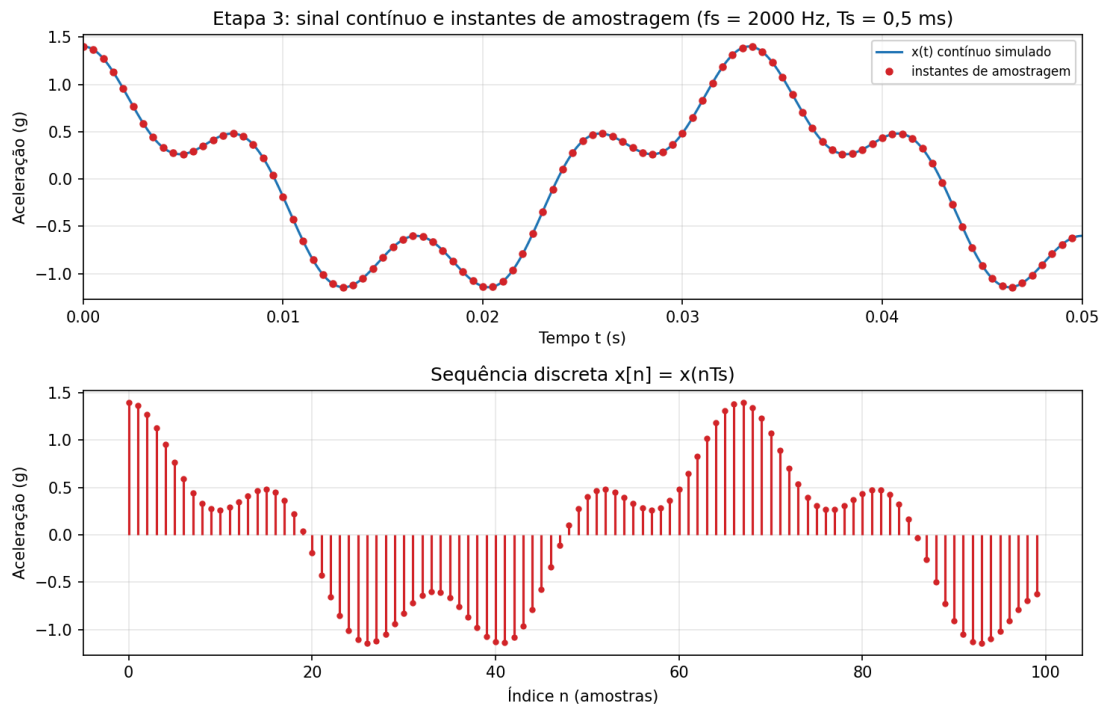


Figura 16. Acima, o sinal contínuo simulado e os instantes de amostragem. Abaixo, a sequência discreta $x[n]$ resultante (primeiras 100 amostras).

A escolha de f_s seguiu o que foi observado no Tópico 2: é a componente mais rápida que define a exigência. Com 16,7 amostras por período de 120 Hz, os “dentes” do sinal ficam bem representados, e os pontos vermelhos da Figura 16 acompanham todos os máximos e mínimos locais.

12.5 Etapa 4 – Quantização

O sinal assume valores positivos e negativos, e por isso foi usado um quantizador bipolar com faixa de -2 g a $+2$ g. A faixa de 4 g é dividida em 2^N níveis:

$$\Delta = \frac{4 \text{ g}}{2^N}, \quad N_1 = 4 \Rightarrow \Delta = 250 \text{ mg}, \quad N_2 = 8 \Rightarrow \Delta = 15,6 \text{ mg}$$

```
# Etapa 4 - Quantização em uma faixa bipolar de -2 g a +2 g
def quantizar(sinal, n_bits, fundo_escala=2.0):
    """Quantizador uniforme entre -fundo_escala e +fundo_escala, com arredondamento."""
    niveis = 2 ** n_bits
    delta = 2 * fundo_escala / niveis
    indice = np.clip(np.round((sinal + fundo_escala) / delta), 0, niveis - 1)
```

```
return indice * delta - fundo_escala, delta, niveis
```

```
x_q4, delta4, niveis4 = quantizar(x, 4)
x_q8, delta8, niveis8 = quantizar(x, 8)
```

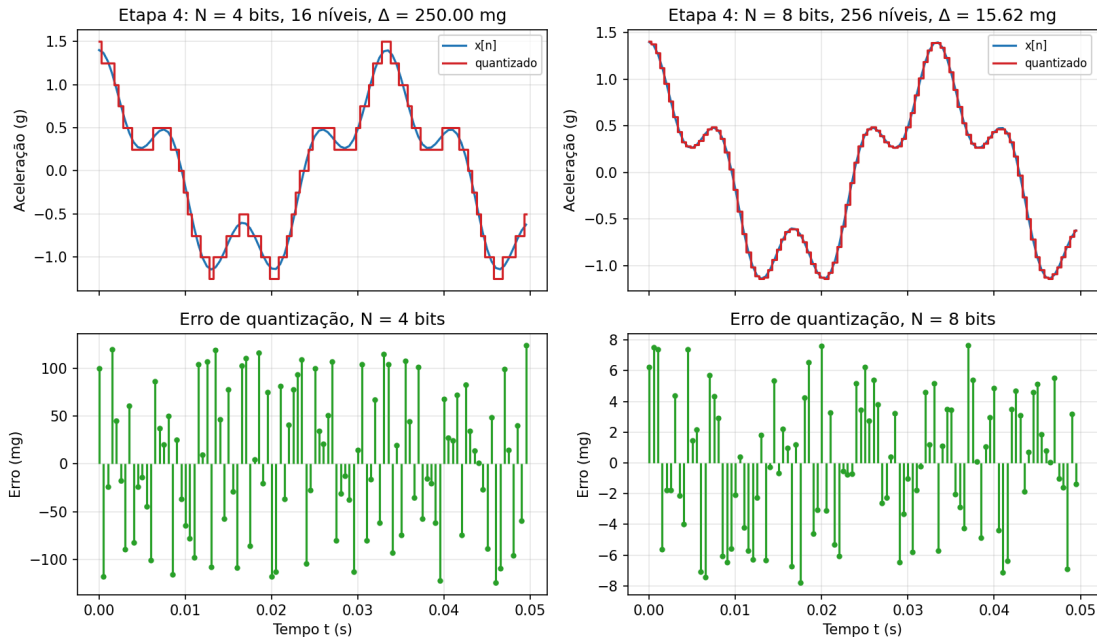


Figura 17. Sinal amostrado e quantizado (acima) e erro de quantização (abaixo) para 4 e 8 bits.

N (bits)	Níveis	Resolução (mg)	Erro máximo (mg)	Erro RMS (mg)	Relação sinal-erro (dB)
4	16	250,0	124,2	75,9	20,0
8	256	15,6	7,8	4,4	44,8

Com 4 bits a resolução de 250 mg é grosseira demais para este sinal. A componente de 120 Hz tem amplitude de 400 mg, menos de dois degraus, e os “dentes” viram patamares quadrados na Figura 17. Com 8 bits o erro máximo cai para 7,8 mg e as curvas se sobrepõem.

Os quatro bits adicionais melhoraram a relação sinal-erro em 24,8 dB, cerca de 6 dB por bit. Isso concorda com o Tópico 3: cada bit divide o erro por 2, e dividir uma amplitude por 2 equivale a 6 dB.

12.6 Etapa 5 – Inclusão de ruído

O sinal contaminado é $x_r[n] = x[n] + r[n]$, em que $r[n]$ é ruído branco gaussiano de média zero e desvio padrão $\sigma = 0,3$ g.

Justificativa do tipo. O ruído gaussiano é o modelo usual para a soma de muitas perturbações pequenas e independentes, como o ruído térmico da eletrônica do sensor e pequenas vibrações aleatórias da estrutura. “Branco” significa que cada amostra de ruído é independente das demais.

Justificativa da amplitude. O valor $\sigma = 0,3$ g corresponde a cerca de 40% do valor eficaz do sinal (0,76 g) e é comparável ao valor eficaz da componente de 120 Hz (0,28 g). É um caso exigente de propósito: o ruído é forte o bastante para esconder a componente menor, o que torna visível o efeito do filtro.

```
# Etapa 5 - Ruído branco gaussiano de média zero
SIGMA = 0.3
gerador = np.random.default_rng(42)
r = gerador.normal(0.0, SIGMA, n.size)
x_r = x + r
snr_entrada = 10 * np.log10(np.mean(x ** 2) / np.mean(r ** 2))
```

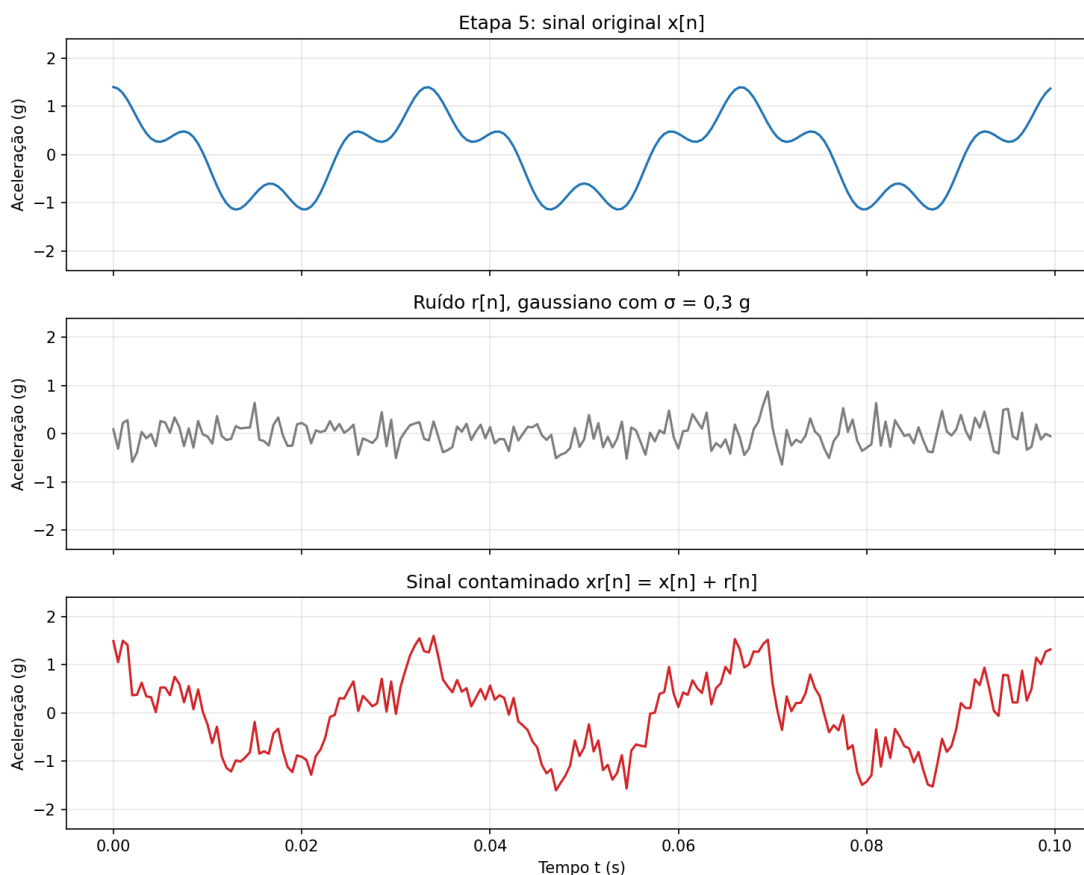


Figura 18. Sinal original $x[n]$, ruído $r[n]$ e sinal contaminado $xr[n]$.

Grandeza	Valor medido
Desvio padrão do ruído	0,3005 g
Potência do sinal	0,5800 g ²
Relação sinal-ruído (SNR)	8,06 dB

A potência medida do sinal (0,5800 g²) coincide com o valor teórico de 0,58 g² calculado na Etapa 1. No sinal contaminado, a oscilação de 30 Hz ainda é reconhecível, mas os “dentes” de 120 Hz desaparecem no meio das variações aleatórias.

12.7 Etapa 6 – Processamento por Sistema LTI

O sinal contaminado foi processado por um filtro de média móvel de M coeficientes iguais. A resposta ao impulso e a saída, calculada por convolução, são:

$$h[n] = \frac{1}{M} \{1, 1, \dots, 1\}, \quad y[n] = x_r[n] * h[n] = \frac{1}{M} \sum_{k=0}^{M-1} x_r[n-k]$$

Cada amostra de saída é a média da amostra atual e das M – 1 anteriores. O filtro usa apenas amostras presentes e passadas, logo é causal. É o mesmo filtro FIR de média móvel estudado no Tópico 10, calculado pela soma de convolução do Tópico 9.

Escolha de M. A escolha envolve um compromisso entre dois efeitos opostos, o mesmo observado no Tópico 10 com M = 3, 5 e 10. A média de M amostras de ruído independente reduz o desvio padrão do ruído para $\sigma/\sqrt{M}$, então M grande remove mais ruído. Por outro lado, a média também suaviza as variações rápidas do próprio sinal, então M grande atenua a componente de 120 Hz. O ganho do filtro em uma frequência f é:

$$|H(f)| = \left| \frac{\text{sen}(\pi f M / f_s)}{M \text{sen}(\pi f / f_s)} \right|$$

Para decidir, foram testados sete valores de M. Para cada um, mediu-se o erro RMS entre a saída e o sinal original sem ruído, já compensando o atraso do filtro.

```
# Etapa 6 - Filtro de média móvel aplicado por convolução
def media_movel(sinal, M):
    """Saída causal y[n] = (x[n] + ... + x[n-M+1]) / M, com o mesmo comprimento da entrada."""
    h = np.ones(M) / M
    return np.convolve(sinal, h)[:sinal.size], h

def ganho(M, f):
    """Módulo da resposta em frequência da média móvel na frequência f."""
    return abs(np.sin(np.pi * f * M / fs) / (M * np.sin(np.pi * f / fs)))

def erro_rms(saida, referencia, M):
    """Erro RMS entre a saída e a referência, compensando o atraso (M-1)/2 e ignorando as bordas."""
    atraso = (M - 1) // 2
    return np.sqrt(np.mean((saida[atraso + M:] - referencia[M:referencia.size - atraso]) ** 2))

candidatos = [3, 5, 7, 9, 11, 15, 21]
erros = []
for M in candidatos:
    y_m, _ = media_movel(x_r, M)
    erros.append(erro_rms(y_m, x, M))

M = candidatos[int(np.argmin(erros))]
atraso = (M - 1) // 2
y, h = media_movel(x_r, M)
```

M	Erro total (g)	Ruído residual (g)	Distorção do sinal (g)	Ganho em 30 Hz	Ganho em 120 Hz
3	0,1795	0,1780	0,0134	0,997	0,953
5	0,1453	0,1385	0,0391	0,991	0,864

M	Erro total (g)	Ruído residual (g)	Distorção do sinal (g)	Ganho em 30 Hz	Ganho em 120 Hz
7	0,1406	0,1170	0,0750	0,982	0,738
9	0,1582	0,1029	0,1182	0,971	0,588
11	0,1904	0,0924	0,1654	0,956	0,425
15	0,2695	0,0785	0,2583	0,919	0,110
21	0,3572	0,0658	0,3528	0,845	0,185

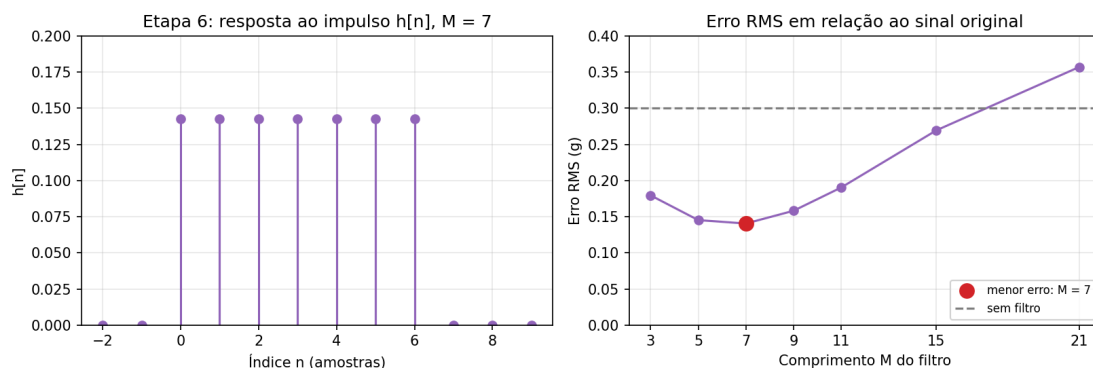


Figura 19. À esquerda, a resposta ao impulso $h[n]$ para $M = 7$. À direita, o erro RMS em relação ao sinal original para cada M . A linha tracejada é o erro sem filtro.

Justificativa. Foi escolhido $M = 7$, o valor de menor erro total (0,1406 g). A tabela mostra o compromisso com clareza: o ruído residual sempre diminui quando M aumenta, mas a distorção do sinal sempre cresce. Até $M = 7$ o ganho com a redução do ruído compensa a distorção. A partir de $M = 9$ a distorção passa a dominar, e com $M = 21$ a saída fica pior do que o sinal sem filtro (0,357 g contra 0,301 g).

A explicação física está no tamanho da janela. Com $M = 7$ a média cobre $7 \times 0,5 \text{ ms} = 3,5 \text{ ms}$. Isso é pouco diante do período de 33,3 ms da componente de 30 Hz, que quase não é afetada. Porém já é 42% do período de 8,33 ms da componente de 120 Hz, que perde parte da amplitude.

O atraso introduzido pelo filtro é de $(M - 1)/2 = 3$ amostras, ou 1,5 ms.

12.8 Etapa 7 – Comparação dos Resultados

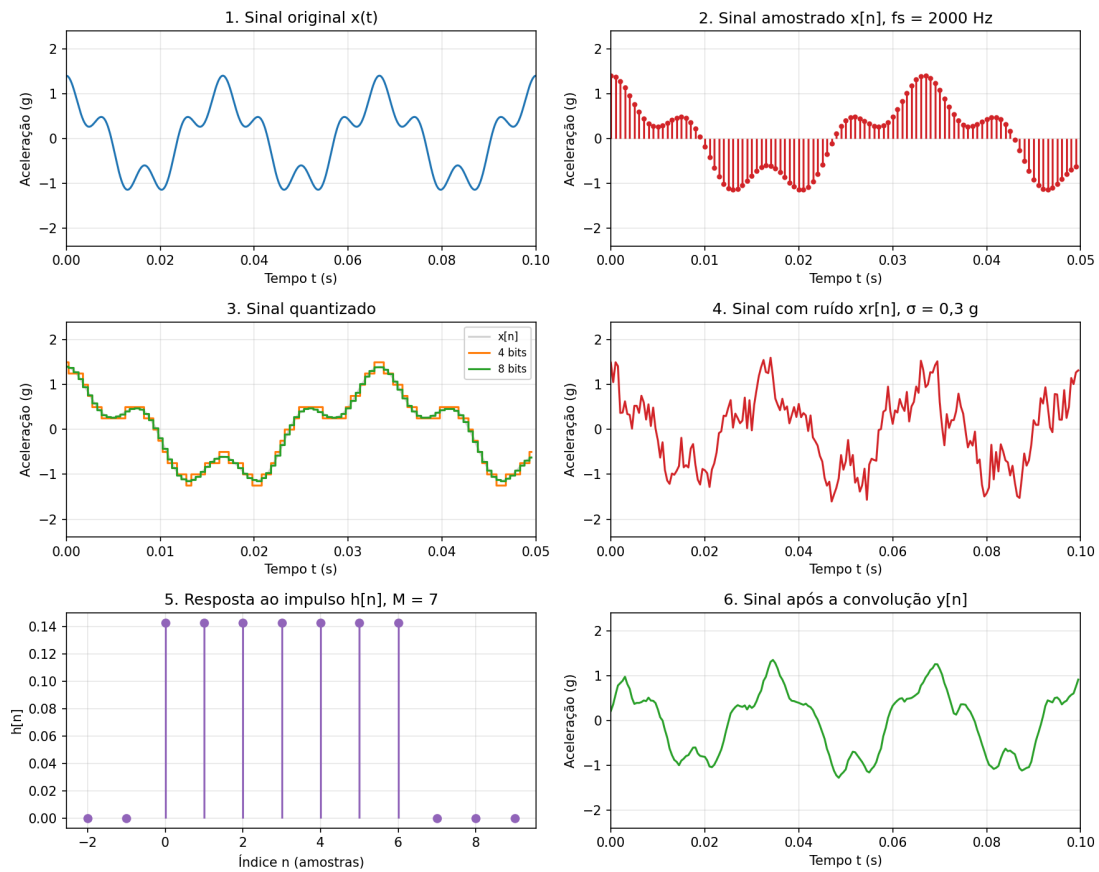


Figura 20. Os estágios da cadeia, na ordem: (1) sinal original, (2) sinal amostrado, (3) sinal quantizado com 4 e 8 bits, (4) sinal com ruído, (5) resposta ao impulso do sistema e (6) sinal após a convolução. Os gráficos de sinal usam a mesma escala vertical.

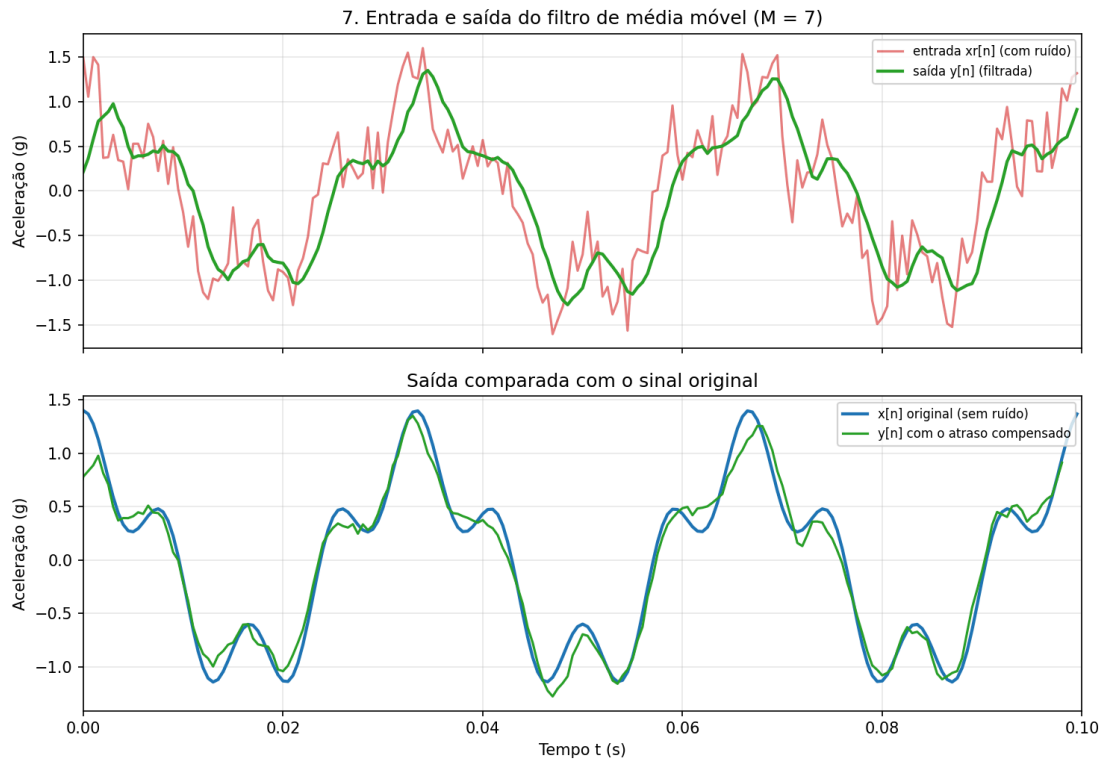


Figura 21. (7) Acima, entrada com ruído e saída do filtro. Abaixo, saída com o atraso de 1,5 ms compensado, comparada com o sinal original sem ruído.

Medida	Antes do filtro	Depois do filtro (M = 7)
Erro RMS em relação ao sinal original	0,3010 g	0,1406 g
Relação sinal-ruído (SNR)	8,06 dB	14,65 dB
Desvio padrão do ruído	0,3005 g	0,1170 g
Amplitude da componente de 30 Hz	1,000 g	0,982 g
Amplitude da componente de 120 Hz	0,400 g	0,295 g

O erro caiu 53,3% e a SNR subiu 6,6 dB. O ruído residual medido (0,1170 g) ficou próximo da previsão teórica $\sigma/\sqrt{M} = 0,3/\sqrt{7} = 0,1134$ g.

12.9 Etapa 8 – Discussão

1. Como o fenômeno foi representado matematicamente? Como a soma de dois cossenos, um para cada causa física de vibração: 30 Hz para a rotação do eixo e 120 Hz para o efeito eletromagnético. Cada amplitude mede a intensidade de uma causa.

2. Como a escolha de f_s modificou a representação do sinal? Com $f_s = 2000$ Hz há 16,7 amostras por período da componente mais rápida, e a sequência reproduz todos os detalhes da forma de onda (Figura 16). Uma f_s muito menor deixaria poucas amostras por “dente”, e, como no Tópico 2, a componente de

120 Hz poderia aparecer como se fosse outra frequência. A f_s também afeta o filtro: o mesmo M cobre um intervalo de tempo diferente se f_s mudar.

3. Qual foi o efeito da quantização? Ela transformou a curva em uma escada e adicionou um erro limitado a meio degrau. Diferentemente do ruído da Etapa 5, esse erro depende do próprio sinal e não pode ser removido depois.

4. Como o número de bits afetou o resultado? Com 4 bits o erro RMS foi de 75,9 mg e os detalhes de 120 Hz ficaram deformados. Com 8 bits o erro caiu para 4,4 mg, cerca de 17 vezes menor, e ficou desprezível diante do ruído de 300 mg. Para este sinal, 8 bits são suficientes: o ruído do sensor, e não o conversor, passa a ser o fator limitante.

5. Qual foi o efeito do ruído? O ruído somou variações aleatórias rápidas ao sinal, reduziu a SNR para 8,06 dB e escondeu a componente de 120 Hz. A oscilação de 30 Hz continuou visível por ter amplitude bem maior que o ruído.

6. Qual a resposta ao impulso do sistema utilizado? $h[n] = 1/7$ para n de 0 a 6 e zero fora desse intervalo (Figura 19). São sete amostras iguais de valor 0,1429, cuja soma é 1. Por isso o filtro não altera o nível médio do sinal. Como $h[n]$ tem duração finita, trata-se de um filtro FIR.

7. Por que o sistema utilizado pode ser considerado LTI? Pelas definições dos Tópicos 7 e 8, ele é linear porque a saída é uma soma de amostras da entrada multiplicadas por constantes: a média de uma combinação de sinais é a mesma combinação das médias. Ele é invariante no tempo porque os coeficientes $1/M$ não dependem de n : atrasar a entrada apenas atrasa a saída. As duas propriedades foram verificadas numericamente. A resposta a $2x[n] + 3r[n]$ foi igual a 2 vezes a resposta a $x[n]$ mais 3 vezes a resposta a $r[n]$, e a resposta à entrada atrasada de 10 amostras foi igual à saída original atrasada de 10 amostras.

8. Qual foi o efeito da convolução? Cada amostra de saída passou a ser a média de 7 amostras vizinhas da entrada. Variações que mudam de sinal dentro da janela, como o ruído, se cancelam em parte. Variações lentas, como a componente de 30 Hz, quase não mudam dentro da janela e passam praticamente intactas. A convolução também atrasou o sinal em 1,5 ms.

9. O sistema reduziu as variações introduzidas pelo ruído? Sim. O desvio padrão do ruído caiu de 0,3005 g para 0,1170 g, uma redução de 61%, próxima do fator teórico $1/\sqrt{7}$. Na Figura 21, a curva de saída é visivelmente mais lisa que a de entrada.

10. Houve alteração significativa do sinal de interesse? Em parte. A componente de 30 Hz foi preservada (ganho de 0,982, perda de 1,8%). A componente de 120 Hz perdeu 26% da amplitude (ganho de 0,738), o que aparece na Figura 21 como “dentes” menos pronunciados na saída. A redução do ruído teve esse preço.

11. Quais limitações foram observadas?

- A média móvel não separa bem sinal e ruído: para remover mais ruído é preciso aceitar mais atenuação da componente de 120 Hz.
- O filtro introduz atraso, e as primeiras $M - 1 = 6$ amostras da saída são um transitório, pois a janela ainda não está cheia.
- O ruído foi somado ao sinal amostrado ideal, como indica o roteiro. Em um sistema real, o ruído entra antes do conversor e é quantizado junto com o sinal.

- O modelo é idealizado: duas senoides de amplitude constante e fase zero, com rotação exata de 1800 rpm. Um motor de indução real gira um pouco abaixo disso e sua vibração tem mais componentes.
- O melhor M foi escolhido comparando a saída com o sinal sem ruído, que em uma medição real não está disponível.

12. Como o modelo poderia ser melhorado?

- Somar o ruído antes da quantização, para reproduzir a ordem real da cadeia de aquisição.
- Incluir harmônicos da rotação e uma componente de defeito de rolamento, com amplitude variável no tempo.
- Trocar a média móvel por um filtro FIR projetado para deixar passar até 120 Hz e rejeitar frequências acima, reduzindo ruído sem atenuar a componente de interesse.
- Analisar o sinal no domínio da frequência, com a Transformada de Fourier da Parte 2, para identificar as componentes de 30 Hz e 120 Hz diretamente, mesmo com ruído.

13. Questões de Síntese

Conceitos Fundamentais e Sinais

1. O que é um sinal?

Um sinal é uma função que transporta informação sobre o estado ou comportamento de um fenômeno físico, variando em função de uma ou mais variáveis independentes (como tempo ou espaço).

2. Como um fenômeno físico pode ser representado matematicamente?

Por meio de funções matemáticas que associam valores da grandeza física (pressão, tensão, temperatura) a variáveis independentes contínuas ou discretas.

3. Qual a diferença entre $x(t)$ e $x[n]$?

- $x(t)$ representa um sinal em **tempo contínuo** (variável $t \in R$ é contínua).
- $x[n]$ representa um sinal em **tempo discreto** (variável $n \in Z$ assume apenas valores inteiros).

Amostragem e Quantização

4. O que significa amostrar um sinal?

É o processo de converter um sinal contínuo no tempo $x(t)$ em uma sequência discreta de pontos $x[n]$, colhendo seus valores em instantes de tempo específicos (geralmente com intervalo constante T_s).

5. Qual a relação entre T_s e f_s ?

Elas são inversamente proporcionais:

$$f_s = \frac{1}{T_s}$$

Onde T_s é o **período de amostragem** (tempo entre duas amostras) e f_s é a **frequência de amostragem** (número de amostras por segundo).

6. Qual a diferença entre amostragem e quantização?

- **Amostragem:** Discretização no **eixo do tempo** (de t para n).
- **Quantização:** Discretização no **eixo da amplitude** (arredondamento dos valores contínuos para um conjunto finito de níveis numéricos).

7. Como o número de bits influencia a representação digital de um sinal?

O número de bits b define o número de níveis de quantização disponíveis ($L = 2^b$). Quanto mais bits:

- Maior a precisão da representação da amplitude;
- Menor o erro de quantização (ruído de quantização);
- Maior a relação sinal-ruído (SNR).

Operações e Tipos de Sinais

8. Qual a importância do impulso discreto $\delta[n]$?

O impulso discreto (delta de Kronecker) é o sinal fundamental em PDS. Ele serve como elemento básico para decompor e representar qualquer sinal discreto e para caracterizar completamente a resposta de um sistema linear e invariante no tempo (LTI).

9. Como funcionam as operações de deslocamento e inversão temporal?

- **Deslocamento temporal** $x[n - n_0]$: Desloca o sinal no tempo. Se $n_0 > 0$, o sinal é atrasado (deslocado para a direita); se $n_0 < 0$, o sinal é adiantado (deslocado para a esquerda).
- **Inversão temporal** $x[-n]$: Espelha o sinal em relação ao eixo vertical ($n = 0$), invertendo a ordem cronológica do tempo.

10. Qual a diferença entre sinais de energia e potência?

- **Sinal de Energia:** Possui energia total finita ($0 < E < \infty$) e potência média nula ($P = 0$). Geralmente são sinais de duração finita.
- **Sinal de Potência:** Possui potência média finita e não nula ($0 < P < \infty$) e energia total infinita ($E = \infty$). Geralmente são sinais periódicos ou infinitos.

Sistemas e Suas Propriedades

11. O que é um sistema?

É um operador físico ou algoritmo que processa um sinal de entrada $x[n]$ para produzir um sinal de saída $y[n]$.

12. O que caracteriza linearidade?

Um sistema é linear se satisfaz os princípios da **superposição** e da **homogeneidade (escalonamento)**:

$$T\{a_1x_1[n] + a_2x_2[n]\} = a_1T\{x_1[n]\} + a_2T\{x_2[n]\}$$

13. O que caracteriza invariância no tempo?

Um sistema é invariante no tempo se um atraso no sinal de entrada produz exatamente o mesmo atraso na resposta do sistema:

$$Sex[n] \rightarrow y[n], \text{então } x[n - n_0] \rightarrow y[n - n_0]$$

14. O que caracteriza causalidade?

Um sistema é causal se a saída no instante presente $y[n_0]$ depende apenas dos valores presentes e passados da entrada ($x[n]$ para $n \leq n_0$), e **não** de valores futuros.

15. O que significa estabilidade BIBO?

BIBO (*Bounded-Input, Bounded-Output*) significa que o sistema produz uma saída limitada para qualquer entrada limitada:

$$\text{Se } |x[n]| \leq M_x < \infty, \text{ então } |y[n]| \leq M_y < \infty$$

Sistemas LTI e Convolução

16. O que é um sistema LTI?

É um sistema que possui simultaneamente as propriedades de **Linearidade** e **Invariância no Tempo** (*Linear Time-Invariant*).

17. O que representa $h[n]$?

Representa a **resposta ao impulso** do sistema — ou seja, é a saída $y[n]$ gerada quando a entrada aplicada é um impulso unitário $x[n] = \delta[n]$.

18. Por que a resposta ao impulso é importante?

Porque ela descreve e caracteriza **completamente** o comportamento dinâmico de um sistema LTI. Conhecendo $h[n]$, é possível calcular a resposta do sistema para qualquer sinal de entrada.

19. O que significa convolução?

É a operação matemática que combina a entrada $x[n]$ e a resposta ao impulso $h[n]$ para determinar a saída $y[n]$ de um sistema LTI.

20. Qual a relação entre $x[n]$, $h[n]$ e $y[n]$?

A relação entre o sinal de entrada $x[n]$, a resposta ao impulso do sistema $h[n]$ e a saída $y[n]$ em um sistema Linear e Invariante no Tempo (LTI) é dada pela **operação de convolução discreta**.

Matematicamente, essa relação é descrita pelo **somatório de convolução**:

$$y[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k]h[n - k]$$

O que essa relação significa na prática?

1. **Decomposição do Sinal:** Qualquer sinal de entrada $x[n]$ pode ser visto como uma soma de vários impulsos discretos $\delta[n]$, cada um ponderado pelo valor da sua amostra $x[k]$.
2. **Superposição de Respostas:** Como o sistema é **Linear e Invariante no Tempo (LTI)**:
 - Cada impulso na entrada gera uma cópia deslocada e escalada da resposta ao impulso $h[n]$.
 - A saída final $y[n]$ é o resultado da **soma (superposição)** de todas essas respostas individuais acumuladas ao longo do tempo.

Em resumo:

- $x[n]$ é o sinal que **entra** no sistema.
- $h[n]$ é a **característica interna** do sistema (a forma como ele reage a um impacto instantâneo).
- $y[n]$ é a **saída** gerada, resultante do cruzamento/combinção (convolução) de $x[n]$ com $h[n]$.

21. Como a convolução pode ser utilizada para modificar um sinal?

Projetando uma resposta ao impulso $h[n]$ adequada, a convolução permite filtrar o sinal — atenuando frequências indesejadas (como ruído) ou realçando componentes específicas (como transições e bordas).

22. Por que um filtro de média móvel tende a suavizar um sinal?

Porque ele calcula a média aritmética de amostras vizinhas. Como o ruído e variações abruptas costumam ter frequências mais altas e variações rápidas de sinal, fazer a média desses pontos atenua essas flutuações rápidas (comportamento de filtro passa-baixas), deixando o sinal mais suave.

14. Conclusões

A Parte 1 percorreu o caminho completo de um sinal, do fenômeno físico até a saída de um sistema LTI, e cada etapa foi confirmada por simulação.

Na representação dos sinais, a amostragem mostrou que a frequência f_s define a fidelidade da sequência discreta: com 8 Hz, uma senoide de 5 Hz gerou as mesmas amostras de uma senoide de 3 Hz. A quantização mostrou que cada bit adicional divide a resolução e o erro por dois. As sequências fundamentais e as operações de deslocamento, inversão, escalonamento e soma formaram a base para o estudo dos sistemas.

Na análise de sinais e sistemas, os cálculos analíticos de energia (2,7778) e de potência média (0,5) coincidiram com os valores numéricos. A análise de cinco sistemas mostrou que memória, causalidade, linearidade, invariância no tempo e estabilidade são propriedades independentes. Para os sistemas LTI, a resposta ao impulso $h[n]$ descreve o sistema por completo, e a saída para qualquer entrada é obtida pela convolução. O cálculo manual de $\{1, 2, 1\} * \{1, 1\} = \{1, 3, 3, 1\}$ coincidiu com o resultado do computador.

O filtro de média móvel reuniu esses conceitos. Tanto no Tópico 10 quanto no mini projeto, aumentar M reduziu o ruído, mas também atrasou o sinal e suavizou as suas variações rápidas. No mini projeto, o valor $M = 7$ reduziu o erro de 0,301 g para 0,141 g, ao custo de atenuar em 26% a componente de 120 Hz.

A principal limitação observada é que a análise no domínio do tempo não separa bem componentes de frequências diferentes. Esse é o ponto de partida da Parte 2, que estuda o domínio da frequência com as transformadas de Fourier, de Laplace e Z.

15. Referências

- SILVA, M. P. *Estudo Dirigido de Processamento Digital de Sinais – Parte 1: Sinais Discretos, Amostragem, Sistemas LTI e Convolução*. Campina Grande: IFPB, 2026.
- OPPENHEIM, A. V.; SCHAFER, R. W. *Discrete-Time Signal Processing*. 3. ed. Pearson, 2010.
- OPPENHEIM, A. V.; WILLSKY, A. S. *Signals and Systems*. 2. ed. Prentice Hall, 1997.
- PROAKIS, J. G.; MANOLAKIS, D. G. *Digital Signal Processing: Principles, Algorithms, and Applications*. 4. ed. Pearson, 2007.
- LATHI, B. P. *Sinais e Sistemas Lineares*. Porto Alegre: Bookman.
- LYONS, R. G. *Understanding Digital Signal Processing*. 3. ed. Pearson, 2011.
- Documentação do NumPy (numpy.org/doc) e do Matplotlib (matplotlib.org).

16. README e Organização dos Arquivos

Os códigos estão organizados na estrutura sugerida pelo roteiro, com um script independente por tópico e um para o mini projeto. As pastas `simulacoes/sistemas/` e `simulacoes/convolucao/` recebem os códigos dos Tópicos 6 a 10, apresentados nas Seções 7 a 11.

```
PDS-Parte-1/
|-- relatorio/                relatório final em PDF
|-- simulacoes/
|   |-- sinais/              Tópicos 1, 4 e 5
|   |-- amostragem/         Tópico 2
|   |-- quantizacao/        Tópico 3
|-- mini-projeto/
|   |-- codigo/              mini_projeto.py
|   |-- dados/              sinais gerados (CSV)
|   |-- figuras/             figuras do mini projeto
|-- resultados/              saídas numéricas das simulações
-- README.md
```

Arquivo	Conteúdo	Figuras geradas
<code>simulacoes/sinais/topico1_sinais.py</code>	Sinais contínuos e discretos	Figura 1
<code>simulacoes/amostragem/topico2_amostragem.py</code>	Amostragem com $f_s = 100, 25$ e 8 Hz	Figura 2
<code>simulacoes/quantizacao/topico3_quantizacao.py</code>	Quantização com 3, 4 e 8 bits	Figura 3
<code>simulacoes/sinais/topico4_sequencias.py</code>	Sequências fundamentais	Figuras 4, 5 e 6
<code>simulacoes/sinais/topico5_operacoes.py</code>	Operações com sinais	Figuras 7 e 8
<code>mini-projeto/codigo/mini_projeto.py</code>	Cadeia completa: vibração de um motor	Figuras 15 a 21

Como executar

Requisitos: Python 3.9 ou superior, NumPy e Matplotlib.

No Google Colab:

1. Abrir um notebook novo em colab.research.google.com.
2. Copiar o conteúdo de um dos arquivos .py para uma célula.
3. Executar a célula. Os gráficos aparecem abaixo dela e também são salvos na pasta `figuras/`.

No computador: cada script é independente e deve ser executado a partir da pasta onde está. O mini projeto é executado a partir da pasta `mini-projeto`.

```
pip install numpy matplotlib
```

```
cd simulacoes/sinais
python topico1_sinais.py
```

```
cd ../../mini-projeto
python codigo/mini_projeto.py
```

Saídas

- Cada script salva as figuras em PNG em uma pasta `figuras/` e imprime os valores numéricos usados nas tabelas deste relatório.
- O mini projeto salva também `dados/sinais_mini_projeto.csv`, com as colunas `n`, `t_s`, `x`, `x_q4`, `x_q8`, `ruído`, `x_r` e `y`.
- O ruído do mini projeto usa semente fixa (42). Os valores numéricos são os mesmos em toda execução.

Apêndice A – Código completo do Mini Projeto

Arquivo `mini-projeto/codigo/mini_projeto.py`:

```
"""Mini Projeto Integrador - aquisição e processamento da vibração de um motor."""
import os
import numpy as np
import matplotlib.pyplot as plt

PASTA = "figuras"
PASTA_DADOS = "dados"
os.makedirs(PASTA, exist_ok=True)
os.makedirs(PASTA_DADOS, exist_ok=True)
plt.rcParams.update({"font.size": 10, "axes.grid": True, "grid.alpha": 0.3})

# Etapa 1 - Modelo: vibração de um motor a 1800 rpm, medida por um acelerômetro (em g)
A1, F1 = 1.0, 30.0
A2, F2 = 0.4, 120.0

def vibracao(t):
    """Componente de rotação (30 Hz) mais componente de 120 Hz."""
    return A1 * np.cos(2 * np.pi * F1 * t) + A2 * np.cos(2 * np.pi * F2 * t)

# Etapa 2 - Geração do sinal "contínuo" em uma grade fina de 100 kHz
DURACAO = 1.0
JANELA = 0.1
t = np.arange(0, DURACAO, 1 / 100_000)
```

```

x_t = vibracao(t)

fig, ax = plt.subplots(figsize=(10, 3.6))
ax.plot(t, x_t, color="tab:blue")
ax.set(title="Etapa 2:  $x(t) = 1,0 \cos(2\pi \cdot 30t) + 0,4 \cos(2\pi \cdot 120t)$ ", xlabel="Tempo t (s)",
        ylabel="Aceleração (g)", xlim=(0, JANELA))
fig.tight_layout()
fig.savefig(os.path.join(PASTA, "mp1_sinal.png"), dpi=150)
plt.show()

# Etapa 3 - Amostragem
fs = 2000.0
Ts = 1 / fs
n = np.arange(0, int(DURACAO * fs))
t_n = n * Ts
x = vibracao(t_n)
visiveis = t_n < 0.05
print(f"Etapa 3: fs = {fs:g} Hz | Ts = {Ts * 1000:g} ms | amostras = {n.size} | "
      f"por período de 30 Hz = {fs / F1:.1f} | por período de 120 Hz = {fs / F2:.1f}")

fig, ax = plt.subplots(2, 1, figsize=(10, 6.5))
ax[0].plot(t, x_t, color="tab:blue", label="x(t) contínuo simulado")
ax[0].plot(t_n[visiveis], x[visiveis], "o", color="tab:red", markersize=4, label="instantes de amostragem")
ax[0].set(title="Etapa 3: sinal contínuo e instantes de amostragem (fs = 2000 Hz, Ts = 0,5 ms)",
          xlabel="Tempo t (s)", ylabel="Aceleração (g)", xlim=(0, 0.05))
ax[0].legend(loc="upper right", fontsize=8)
ax[1].stem(n[visiveis], x[visiveis], linefmt="tab:red", markerfmt=".", basefmt=" ")
ax[1].set(title="Sequência discreta  $x[n] = x(nTs)$ ", xlabel="Índice n (amostras)", ylabel="Aceleração (g)")
fig.tight_layout()
fig.savefig(os.path.join(PASTA, "mp2_amostragem.png"), dpi=150)
plt.show()

# Etapa 4 - Quantização em uma faixa bipolar de -2 g a +2 g
def quantizar(sinal, n_bits, fundo_escala=2.0):
    """Quantizador uniforme entre -fundo_escala e +fundo_escala, com arredondamento."""
    niveis = 2 ** n_bits
    delta = 2 * fundo_escala / niveis
    indice = np.clip(np.round((sinal + fundo_escala) / delta), 0, niveis - 1)
    return indice * delta - fundo_escala, delta, niveis

fig, ax = plt.subplots(2, 2, figsize=(11, 6.5), sharex=True)
quantizados = {}
for coluna, n_bits in enumerate([4, 8]):
    x_q, delta, niveis = quantizar(x, n_bits)
    quantizados[n_bits] = x_q
    erro = x_q - x
    erro_rms = np.sqrt(np.mean(erro ** 2))
    sqnr = 10 * np.log10(np.mean(x ** 2) / np.mean(erro ** 2))
    print(f"Etapa 4: N = {n_bits} bits | niveis = {niveis} | resolução = {delta:.6f} g | "
          f"erro máx = {np.max(np.abs(erro)):.5f} g | erro RMS = {erro_rms:.5f} g | SQNR = {sqnr:.1f} dB")
    ax[0, coluna].plot(t_n[visiveis], x[visiveis], color="tab:blue", label="x[n]")
    ax[0, coluna].step(t_n[visiveis], x_q[visiveis], where="mid", color="tab:red", label="quantizado")
    ax[0, coluna].set(title=f"Etapa 4: N = {n_bits} bits, {niveis} níveis,  $\Delta = \{delta * 1000:.2f\}$  mg",
                      ylabel="Aceleração (g)")
    ax[0, coluna].legend(loc="upper right", fontsize=8)
    ax[1, coluna].stem(t_n[visiveis], erro[visiveis] * 1000, linefmt="tab:green", markerfmt=".", basefmt=" ")
    ax[1, coluna].set(title=f"Erro de quantização, N = {n_bits} bits", xlabel="Tempo t (s)", ylabel="Erro (mg)")
fig.tight_layout()
fig.savefig(os.path.join(PASTA, "mp3_quantizacao.png"), dpi=150)
plt.show()

# Etapa 5 - Ruído branco gaussiano de média zero
SIGMA = 0.3
gerador = np.random.default_rng(42)

```

```

r = gerador.normal(0.0, SIGMA, n.size)
x_r = x + r
snr_entrada = 10 * np.log10(np.mean(x ** 2) / np.mean(r ** 2))
print(f"Etapa 5: desvio padrão do ruído = {np.std(r):.4f} g | potência do sinal = {np.mean(x ** 2):.4f} g²
| "
      f"SNR = {snr_entrada:.2f} dB")

janela_n = t_n < JANELA
fig, ax = plt.subplots(3, 1, figsize=(10, 8), sharex=True)
ax[0].plot(t_n[janela_n], x[janela_n], color="tab:blue")
ax[0].set(title="Etapa 5: sinal original x[n]", ylabel="Aceleração (g)")
ax[1].plot(t_n[janela_n], r[janela_n], color="gray")
ax[1].set(title="Ruído r[n], gaussiano com  $\sigma = 0,3$  g", ylabel="Aceleração (g)")
ax[2].plot(t_n[janela_n], x_r[janela_n], color="tab:red")
ax[2].set(title="Sinal contaminado x_r[n] = x[n] + r[n]", xlabel="Tempo t (s)", ylabel="Aceleração (g)")
for eixo in ax:
    eixo.set_ylim(-2.4, 2.4)
fig.tight_layout()
fig.savefig(os.path.join(PASTA, "mp4_ruído.png"), dpi=150)
plt.show()

# Etapa 6 - Filtro de média móvel aplicado por convolução
def media_movel(sinal, M):
    """Saída causal y[n] = (x[n] + ... + x[n-M+1]) / M, com o mesmo comprimento da entrada."""
    h = np.ones(M) / M
    return np.convolve(sinal, h)[:sinal.size], h

def ganho(M, f):
    """Módulo da resposta em frequência da média móvel na frequência f."""
    return abs(np.sin(np.pi * f * M / fs) / (M * np.sin(np.pi * f / fs)))

def erro_rms(saida, referencia, M):
    """Erro RMS entre a saída e a referência, compensando o atraso (M-1)/2 e ignorando as bordas."""
    atraso = (M - 1) // 2
    return np.sqrt(np.mean((saida[atraso + M:] - referencia[M:referencia.size - atraso]) ** 2))

candidatos = [3, 5, 7, 9, 11, 15, 21]
erros = []
print("Etapa 6: M | erro total | ruído residual | distorção | ganho 30 Hz | ganho 120 Hz")
for M in candidatos:
    y_m, _ = media_movel(x_r, M)
    sinal_filtrado, _ = media_movel(x, M)
    ruido_filtrado, _ = media_movel(r, M)
    erros.append(erro_rms(y_m, x, M))
    print(f"
          {M:2d} | {erros[-1]:10.4f} | {np.std(ruido_filtrado[M:]):14.4f} | "
          f"{erro_rms(sinal_filtrado, x, M):9.4f} | {ganho(M, F1):11.4f} | {ganho(M, F2):.4f}")

M = candidatos[int(np.argmax(erros))]
atraso = (M - 1) // 2
y, h = media_movel(x_r, M)
print(f"Etapa 6: M escolhido = {M} | atraso = {atraso} amostras = {atraso * Ts * 1000:.1f} ms | "
      f"soma de h[n] = {h.sum():.4f}")

fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
ax[0].stem(np.arange(-2, M + 3), np.r_[0, 0, h, 0, 0, 0], linefmt="tab:purple", markerfmt="o", basefmt="")
ax[0].set(title=f"Etapa 6: resposta ao impulso h[n], M = {M}", xlabel="Índice n (amostras)",
          ylabel="h[n]", ylim=(0, 0.2))
ax[1].plot(candidatos, erros, "o-", color="tab:purple")
ax[1].plot(M, min(erros), "o", color="tab:red", markersize=10, label=f"menor erro: M = {M}")
ax[1].axhline(np.std(r), color="gray", linestyle="--", label="sem filtro")
ax[1].set(title="Erro RMS em relação ao sinal original", xlabel="Comprimento M do filtro",
          ylabel="Erro RMS (g)", xticks=candidatos, ylim=(0, 0.4))
ax[1].legend(loc="lower right", fontsize=8)
fig.tight_layout()

```

```

fig.savefig(os.path.join(PASTA, "mp5_filtro.png"), dpi=150)
plt.show()

# Verificação da linearidade e da invariância no tempo do filtro
x_a, x_b = x, r
combinado, _ = media_movel(2 * x_a + 3 * x_b, M)
separado = 2 * media_movel(x_a, M)[0] + 3 * media_movel(x_b, M)[0]
print("Etapa 8: superposição verificada:", np.allclose(combinado, separado))
deslocado = np.r_[np.zeros(10), x_r[:-10]]
print("Etapa 8: invariância no tempo verificada:",
      np.allclose(media_movel(deslocado, M)[0][10:], y[:-10]))

# Etapa 7 - Métricas finais
y_alinhado = y[atraso + M:]
referencia = x[M:x.size - atraso]
erro_antes = np.sqrt(np.mean((x_r - x) ** 2))
erro_depois = np.sqrt(np.mean((y_alinhado - referencia) ** 2))
snr_saida = 10 * np.log10(np.mean(referencia ** 2) / np.mean((y_alinhado - referencia) ** 2))
ruído_saida = np.std(media_movel(r, M)[0][M:])
print(f"Etapa 7: erro RMS antes = {erro_antes:.4f} g | depois = {erro_depois:.4f} g | "
      f"redução = {(1 - erro_depois / erro_antes) * 100:.1f} %")
print(f"Etapa 7: SNR antes = {snr_entrada:.2f} dB | depois = {snr_saida:.2f} dB")
print(f"Etapa 7: ruído residual = {ruído_saida:.4f} g | previsão sigma/raiz(M) = {SIGMA / np.sqrt(M):.4f} g")
print(f"Etapa 7: ganho em 30 Hz = {ganho(M, F1):.4f} | ganho em 120 Hz = {ganho(M, F2):.4f}")

fig, ax = plt.subplots(3, 2, figsize=(12, 9.5))
ax[0, 0].plot(t, x_t, color="tab:blue")
ax[0, 0].set(title="1. Sinal original x(t)", xlim=(0, JANELA))
ax[0, 1].stem(t_n[visiveis], x[visiveis], linefmt="tab:red", markerfmt=" ", basefmt=" ")
ax[0, 1].set(title="2. Sinal amostrado x[n], fs = 2000 Hz", xlim=(0, 0.05))
ax[1, 0].plot(t_n[visiveis], x[visiveis], color="lightgray", label="x[n]")
ax[1, 0].step(t_n[visiveis], quantizados[4][visiveis], where="mid", color="tab:orange", label="4 bits")
ax[1, 0].step(t_n[visiveis], quantizados[8][visiveis], where="mid", color="tab:green", label="8 bits")
ax[1, 0].set(title="3. Sinal quantizado", xlim=(0, 0.05))
ax[1, 0].legend(loc="upper right", fontsize=8)
ax[1, 1].plot(t_n[janela_n], x_r[janela_n], color="tab:red")
ax[1, 1].set(title="4. Sinal com ruído xr[n], σ = 0,3 g", xlim=(0, JANELA))
ax[2, 0].stem(np.arange(-2, M + 3), np.r_[0, 0, h, 0, 0, 0], linefmt="tab:purple", markerfmt="o",
basefmt=" ")
ax[2, 0].set(title=f"5. Resposta ao impulso h[n], M = {M}", xlabel="Índice n (amostras)", ylabel="h[n]")
ax[2, 1].plot(t_n[janela_n], y[janela_n], color="tab:green")
ax[2, 1].set(title="6. Sinal após a convolução y[n]", xlabel="Tempo t (s)", xlim=(0, JANELA))
for eixo in [ax[0, 0], ax[0, 1], ax[1, 0], ax[1, 1], ax[2, 1]]:
    eixo.set_ylabel("Aceleração (g)")
    eixo.set_ylim(-2.4, 2.4)
for eixo in [ax[0, 0], ax[0, 1], ax[1, 0], ax[1, 1]]:
    eixo.set_xlabel("Tempo t (s)")
fig.tight_layout()
fig.savefig(os.path.join(PASTA, "mp6_estagios.png"), dpi=150)
plt.show()

fig, ax = plt.subplots(2, 1, figsize=(10, 7), sharex=True)
ax[0].plot(t_n[janela_n], x_r[janela_n], color="tab:red", alpha=0.6, label="entrada xr[n] (com ruído)")
ax[0].plot(t_n[janela_n], y[janela_n], color="tab:green", linewidth=2, label="saída y[n] (filtrada)")
ax[0].set(title=f"7. Entrada e saída do filtro de média móvel (M = {M})", ylabel="Aceleração (g)")
ax[0].legend(loc="upper right", fontsize=8)
ax[1].plot(t_n[janela_n], x[janela_n], color="tab:blue", linewidth=2, label="x[n] original (sem ruído)")
ax[1].plot(t_n[janela_n] - atraso * Ts, y[janela_n], color="tab:green", label="y[n] com o atraso compensado")
ax[1].set(title="Saída comparada com o sinal original", xlabel="Tempo t (s)", ylabel="Aceleração (g)",
      xlim=(0, JANELA))
ax[1].legend(loc="upper right", fontsize=8)
fig.tight_layout()
fig.savefig(os.path.join(PASTA, "mp7_entrada_saida.png"), dpi=150)
plt.show()

# Dados da simulação salvos para conferência
np.savetxt(os.path.join(PASTA_DADOS, "sinais_mini_projeto.csv"),

```

```
np.column_stack([n, t_n, x, quantizados[4], quantizados[8], r, x_r, y]), delimiter=",",
header="n,t_s,x,x_q4,x_q8,ruido,x_r,y", comments="", fmt="%.6f")
```